

```

    searchEntries,
    sortHits,
    type SearchContext,
  } from "../src/lib/whiteboard/search";
import type { Board, Folder, WhiteboardElement } from "../src/lib/whiteboard/types";
import { getPages } from "../src/lib/whiteboard/notebook";
import { useWhiteboard } from "../src/store/whiteboard";

const NL = String.fromCharCode(10);
let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`      ${err as Error}.message`);
  }
}

function folder(id: string, name: string, parentId: string | null): Folder {
  return { id, name, parentId, expanded: true, createdAt: 1 };
}

function text(id: string, body: string, y = 0, updatedAt = 1): WhiteboardElement {
  return {
    id, type: "text", x: 0, y, width: 100, height: 20, rotation: 0, opacity: 1,
    createdAt: 1, updatedAt, text: body, color: "#000", fontSize: 16,
    fontFamily: "inter", fontWeight: 400, align: "left",
  } as WhiteboardElement;
}

function sticky(id: string, body: string, y = 0): WhiteboardElement {
  return {
    id, type: "sticky", x: 0, y, width: 100, height: 100, rotation: 0, opacity: 1,
    createdAt: 1, updatedAt: 1, text: body, color: "yellow", fontSize: 16, align: "left",
  } as WhiteboardElement;
}

function stroke(id: string): WhiteboardElement {
  return {
    id, type: "stroke", x: 0, y: 0, rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1,
    points: [[0, 0, 0.5], [10, 10, 0.5]], color: "#000", size: 3,
  } as unknown as WhiteboardElement;
}

function canvas(id: string, name: string, folderId: string | null, elements: WhiteboardElement[], updatedAt = 1): Board {
  return {
    id, name, folderId, elements, camera: { x: 0, y: 0, zoom: 1 },
    createdAt: 1, updatedAt,
  };
}

function notebook(id: string, name: string, folderId: string | null, pages: WhiteboardElement[][]): Board {
  return {
    id, name, folderId, elements: [], camera: { x: 0, y: 0, zoom: 1 },
    createdAt: 1, updatedAt: 1, kind: "notebook",
    pages: pages.map((els, i) => ({ id: `${id}-p${i}`, elements: els, createdAt: 1, updatedAt: 1 })),
    activePageId: `${id}-p0`,
  };
}

// Uni ■■■ Physics ■■ Labs
//      ■■ Maths
// Old (top level)
const folders = [
  folder("uni", "Uni", null),
  folder("phys", "Physics", "uni"),
  folder("labs", "Labs", "phys"),
  folder("maths", "Maths", "uni"),
  folder("old", "Old", null),
];

const boards: Board[] = [

```

```

notebook("lect", "Lecture notes", "phys", [
  [text("a", "Momentum is conserved", 300)],
  [sticky("b", "***Exam** momentum question", 50), stroke("ink")],
  [text("c", "momentum again", 10)],
]),
canvas("hw", "Homework", "phys", [text("d", "momentum homework")], 5),
canvas("lab1", "Lab 1", "labs", [text("e", "measure momentum")]),
canvas("alg", "Algebra", "maths", [text("f", "momentum of a group?")]),
canvas("arch", "Archive", "old", [text("g", "momentum archive", 0, 99)], 99),
canvas("mom", "Momentum", "old", []),
];

const ctx: SearchContext = { activeBoardId: "lect", folderId: "phys", folders };
const index = buildSearchIndex(boards, folders);

console.log(NL + "■■■ Search scopes & ranking ■■■");

test("'This note' finds only the open note's content, in page order", () => {
  const hits = searchEntries(index, "momentum", "note", ctx);
  assert.deepEqual(hits.map((h) => h.entry.elementId), ["a", "b", "c"]);
  assert.deepEqual(hits.map((h) => h.entry.pageIndex), [0, 1, 2]);
});

test("'This folder' covers the folder and its sub-folders only", () => {
  const ids = searchEntries(index, "momentum", "folder", ctx).map((h) => h.entry.boardId);
  assert.ok(ids.includes("lect") && ids.includes("hw") && ids.includes("lab1"));
  assert.ok(!ids.includes("alg"), "a sibling folder is outside");
  assert.ok(!ids.includes("arch") && !ids.includes("mom"), "another tree is outside");
});

test("'Everywhere' is nearest first: note, folder, around it, elsewhere", () => {
  const hits = searchEntries(index, "momentum", "everywhere", ctx);
  const groups = groupHits(hits, "everywhere", ctx);
  assert.deepEqual(groups.map((g) => g.id), ["note", "folder", "near", "elsewhere"]);
  assert.deepEqual(groups.map((g) => g.label), ["In this note", "In Physics", "Sub-folders & parent", "Elsewhere"]);
  assert.deepEqual(groups[1].hits.map((h) => h.entry.boardId), ["hw"]);
  assert.deepEqual(groups[2].hits.map((h) => h.entry.boardId), ["lab1"]);
  // A note NAMED "Momentum" is the best match of all, yet it is far away, so
  // it still comes after everything nearby - then leads its own group.
  assert.deepEqual(groups[3].hits.map((h) => h.entry.id), ["board:mom", "arch:g", "alg:f"]);
});

test("the parent folder counts as near; a sibling does not", () => {
  const inMaths = { ...ctx, activeBoardId: "alg", folderId: "maths" };
  const hits = searchEntries(index, "momentum", "everywhere", inMaths);
  const near = hits.filter((h) => h.nearness === "near").map((h) => h.entry.boardId);
  assert.deepEqual(near, [], "Uni itself holds no notes, and Physics is a sibling");
  const physics = hits.find((h) => h.entry.boardId === "hw");
  assert.equal(physics?.nearness, "elsewhere");
  const fromLabs = searchEntries(index, "homework", "everywhere", { ...ctx, activeBoardId: "lab1", folderId: "labs" });
  assert.equal(fromLabs[0]?.nearness, "near", "Physics is the parent of Labs");
});

test("within a group: title beats whole word beats prefix beats substring", () => {
  const title = matchEntry(index.find((e) => e.id === "board:mom")!, ["momentum"]);
  const word = matchEntry(index.find((e) => e.id === "arch:g")!, ["momentum"]);
  const idx = buildSearchIndex(
    [canvas("x", "X", null, [text("p", "momentums"), text("s", "antimomentum"), text("w", "the momentum")])],
    []
  );
  const q = (id: string) => matchEntry(idx.find((e) => e.id === `x:${id}`)!, ["momentum"]).score;
  assert.ok(title.score > word.score, "title first");
  assert.ok(q("w") > q("p") && q("p") > q("s"), "word > prefix > substring");
});

test("equal matches fall back to the most recently edited", () => {
  const idx = buildSearchIndex(
    [canvas("x", "X", null, [text("old", "alpha", 0, 10), text("new", "alpha", 50, 20)])],
    []
  );
  const hits = searchEntries(idx, "alpha", "everywhere", { activeBoardId: null, folderId: null, folders: [] });
  assert.deepEqual(hits.map((h) => h.entry.elementId), ["new", "old"]);
});

test("every word must match; the whole phrase is highlighted as one", () => {

```

```

    assert.equal(searchEntries(index, "momentum banana", "everywhere", ctx).length, 0);
    const hit = searchEntries(index, "exam momentum", "note", ctx)[0];
    assert.equal(hit.entry.elementId, "b");
    const parts = highlightParts(makeSnippet(hit)).filter((p) => p.hit).map((p) => p.text);
    assert.deepEqual(parts, ["Exam momentum"]);
  });

test("markdown is searched as displayed; ink is never a result", () => {
  assert.equal(plainText("## **Bold** [[Target|alias]] ==hi=="), "Bold alias hi");
  assert.ok(!index.some((e) => e.elementId === "ink"));
});

test("case, accents and Hebrew niqqud are ignored, highlight lands on the written text", () => {
  assert.equal(foldText("CAFÉ"), "cafe");
  const idx = buildSearchIndex([canvas("h", "H", null, [text("t", "■■■■ ■■■■■■ ■■■■■")])], []);
  const hit = searchEntries(idx, "■■■■", "everywhere", { activeBoardId: null, folderId: null, folders: [] })[0];
  assert.ok(hit, "found without niqqud");
  const marked = highlightParts(makeSnippet(hit)).find((p) => p.hit)?.text;
  assert.equal(marked, "■■■■■■■■");
});

test("folder names, note names and tags are found in the wide scopes, not in 'This note'", () => {
  const tagged = { ...canvas("t", "Tagged", "phys", []), tags: ["exam"] };
  const idx = buildSearchIndex([...boards, tagged, folders]);
  const kinds = searchEntries(idx, "exam", "everywhere", ctx).map((h) => h.entry.kind);
  assert.ok(kinds.includes("tag"));
  const labs = searchEntries(idx, "labs", "everywhere", ctx);
  assert.equal(labs[0].entry.kind, "folder");
  assert.ok(searchEntries(idx, "lecture", "note", ctx).length === 0, "the note's own name is not 'in' it");
});

test("a long text is cut to a window around the match", () => {
  const long = "word ".repeat(60) + "needle " + "tail ".repeat(60);
  const idx = buildSearchIndex([canvas("l", "L", null, [text("t", long)])], []);
  const hit = searchEntries(idx, "needle", "everywhere", { activeBoardId: null, folderId: null, folders: [] })[0];
  const snip = makeSnippet(hit, 60);
  assert.ok(snip.text.startsWith("...") && snip.text.endsWith("..."));
  assert.ok(snip.text.length <= 62);
  const [at, len] = snip.ranges[0];
  assert.equal(snip.text.slice(at, at + len), "needle");
});

test("PDF text hits take the note's place and sort by page within 'This note'", () => {
  const info = scopeInfo(ctx);
  const lect = boards[0];
  const p = pdfHit(lect, { pageIndex: 1, snippet: "the momentum of a body", matchStart: 4, matchLength: 8 }, "momentum", 0, info);
  assert.equal(p.nearness, "note");
  assert.equal(p.entry.pageId, "lect-p1", "page id filled in from the index");
  const merged = sortHits([...searchEntries(index, "momentum", "note", ctx), p], "note");
  assert.deepEqual(merged.map((h) => h.entry.elementId ?? "pdf"), ["a", "pdf", "b", "c"]);
  const marked = highlightParts(makeSnippet(p)).find((x) => x.hit)?.text;
  assert.equal(marked, "momentum");
});

test("at the top level, 'This folder' means the loose top-level notes", () => {
  const top = [canvas("loose", "Loose", null, [text("z", "zeta")]), canvas("in", "In", "uni", [text("y", "zeta")])];
  const idx = buildSearchIndex(top, folders);
  const hits = searchEntries(idx, "zeta", "folder", { activeBoardId: "loose", folderId: null, folders });
  assert.deepEqual(hits.map((h) => h.entry.boardId), ["loose"]);
});

test("re-indexing reuses unchanged boards", () => {
  const again = buildSearchIndex(boards, folders);
  const a = index.find((e) => e.id === "hw:d");
  const b = again.find((e) => e.id === "hw:d");
  assert.equal(a, b, "same entry object: the board was not re-read");
});

test("opening a result on another notebook page turns to it and centres it, unselected", () => {
  const S = () => useWhiteboard.getState();
  const id = S().createNotebook("Find me", null);
  S().setActiveBoard(id);
  S().addPage(); // page 2 is now active
  S().addElement(text("far-note", "deep in the notebook", 400));

```

```

const page2 = S().boards.find((b) => b.id === id)!.activePageId;
S().goToPage(getPages(S().boards.find((b) => b.id === id))[0].id, 800);

S().focusElement(id, "far-note", { select: false });
const after = S();
assert.equal(after.boards.find((b) => b.id === id)!.activePageId, page2, "turned to the element's page");
assert.equal(after.camera.x, 50);
assert.equal(after.camera.y, 410, "centred on the element in page coordinates");
assert.deepEqual(after.selectedIds, [], "search shows, it does not select");
});

console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/search_scopes.test.ts ===
=====

=====
// === START FILE: tests/select_modes.test.ts ===
// File 177 of 194 | Lines: 199 | Size: 5801 bytes
=====

/**
 * Select modes: loose lasso vs strict Samsung-style encircle.
 *
 * Loose (any visible point inside) is fast; strict (EVERY point inside a
 * CLOSED loop) is surgical. A neighbour the loop only grazes must stay
 * unselected in strict mode, and an open squiggle must select nothing.
 */

import assert from "node:assert/strict";
import {
  isElementFullyInPolygon,
  isElementInPolygon,
  isPointInPolygon,
} from "../src/lib/whiteboard/geometry";
import type { LineElement, RectangleElement, StrokeElement } from "../src/lib/whiteboard/types";

const NL = String.fromCharCode(10);

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(` ${err as Error}.message`);
  }
}

let seq = 0;
function rect(x: number, y: number, w: number, h: number): RectangleElement {
  seq++;
  return {
    id: `r${seq}`,
    type: "rectangle",
    x,
    y,
    width: w,
    height: h,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    fillColor: "transparent",
    fillStyle: "transparent",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  };
}

```

```

function stroke(pts: Array<[number, number]>, ox = 0, oy = 0): StrokeElement {
  seq++;
  return {
    id: `s${seq}`,
    type: "stroke",
    x: ox,
    y: oy,
    rotation: 0,
    opacity: 1,
    points: pts.map(([x, y]) => [x, y, 0.5] as [number, number, number]),
    color: "#000",
    strokeWidth: 3,
    strokeStyle: "pen",
    createdAt: 1,
    updatedAt: 1,
  };
}

/** Big square loop around (0,0)-(200,200). */
const LOOP = [
  { x: 0, y: 0 },
  { x: 200, y: 0 },
  { x: 200, y: 200 },
  { x: 0, y: 200 },
];

function main() {
  console.log(NL + "■■ Select modes ■■");

  test("loose lasso takes a fully enclosed box", () => {
    assert.equal(isElementInPolygon(rect(50, 50, 40, 40), LOOP), true);
  });

  test("loose lasso takes a box the loop only grazes", () => {
    assert.equal(isElementInPolygon(rect(180, 180, 60, 60), LOOP), true);
  });

  test("strict encircle takes the fully enclosed box", () => {
    assert.equal(isElementFullyInPolygon(rect(50, 50, 40, 40), LOOP), true);
  });

  test("strict encircle refuses the grazed neighbour", () => {
    assert.equal(isElementFullyInPolygon(rect(180, 180, 60, 60), LOOP), false);
  });

  test("strict encircle refuses an outside box", () => {
    assert.equal(isElementFullyInPolygon(rect(300, 300, 40, 40), LOOP), false);
  });

  test("both modes need at least a triangle", () => {
    const line = [
      { x: 0, y: 0 },
      { x: 200, y: 200 },
    ];
    assert.equal(isElementInPolygon(rect(50, 50, 40, 40), line), false);
    assert.equal(isElementFullyInPolygon(rect(50, 50, 40, 40), line), false);
  });

  // ---- handwriting: the case the strict mode exists for -----
  // A stroke's bounding box is a rectangle around a curve, so for anything
  // diagonal the box corners sit in empty space. Testing THOSE made strict
  // encircle fail on every hugging loop – the Samsung gesture itself.

  test("strict encircle takes a stroke from a loop that hugs the ink", () => {
    // Diagonal ink from (10,10) to (90,90); the loop follows it closely, so
    // the stroke's box corners (10,90) and (90,10) are OUTSIDE the loop.
    const ink = stroke([
      [0, 0],
      [20, 20],
      [40, 40],
      [60, 60],
      [80, 80],
    ], 10, 10);
    const hug = [
      { x: 0, y: 20 },
      { x: 80, y: 100 },
      { x: 100, y: 100 },

```

```

    { x: 100, y: 80 },
    { x: 20, y: 0 },
    { x: 0, y: 0 },
  ];
  assert.equal(isPointInPolygon(10, 90, hug), false, "the box corner really is outside");
  assert.equal(isElementFullyInPolygon(ink, hug), true, "but every point of ink is inside");
});

test("strict encircle lifts one line of writing and leaves its neighbour", () => {
  const above = stroke([[10, 10], [30, 14], [50, 10], [70, 14]]);
  const below = stroke([[10, 40], [30, 44], [50, 40], [70, 44]]);
  // A loop snaked between the two lines, around the lower one only.
  const snake = [
    { x: 5, y: 28 },
    { x: 80, y: 28 },
    { x: 80, y: 55 },
    { x: 5, y: 55 },
  ];
  assert.equal(isElementFullyInPolygon(below, snake), true, "the encircled word comes");
  assert.equal(isElementFullyInPolygon(above, snake), false, "the line above stays put");
});

test("strict refuses a half-covered word where loose takes it", () => {
  const word = stroke([[10, 40], [30, 44], [50, 40], [70, 44]]);
  const half = [
    { x: 5, y: 28 },
    { x: 40, y: 28 },
    { x: 40, y: 55 },
    { x: 5, y: 55 },
  ];
  assert.equal(isElementInPolygon(word, half), true, "loose is greedy – that is its job");
  assert.equal(isElementFullyInPolygon(word, half), false, "strict wants all of it");
});

test("a diagonal line is its two ends, not the air around them", () => {
  const diag: LineElement = {
    id: "ln1",
    type: "line",
    x: 20,
    y: 20,
    x2: 60,
    y2: 60,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  };
  // A narrow band along the diagonal: both ENDS are inside it, both box
  // corners are not.
  const hug = [
    { x: 0, y: 20 },
    { x: 90, y: 110 },
    { x: 110, y: 90 },
    { x: 20, y: 0 },
  ];
  assert.equal(isPointInPolygon(20, 80, hug), false, "the box corner really is outside");
  assert.equal(isElementFullyInPolygon(diag, hug), true, "but both ends are inside");
});

console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);
}

```

```
void main();
```

```

=====
// === END FILE: tests/select_modes.test.ts ===
=====

```

```

=====
// === START FILE: tests/shape_recognition.test.ts ===
// File 178 of 194 | Lines: 408 | Size: 16276 bytes
=====

```

```

/**
 * Shape recognition on hand-drawn strokes.
 *
 * Every shape is drawn many times the way a hand draws it - a slow wobble, a
 * little jitter, starting anywhere, closing with a small gap or an overshoot,
 * at any angle - and must come out as the shape meant, the same size and in
 * the same place. Things that are NOT shapes (handwriting, an open arc, a
 * zig-zag, a scribble) must be left exactly as drawn.
 */

import assert from "node:assert/strict";
import fs from "node:fs";
import path from "node:path";
import { joinLine, recognizeShape, type ShapeKind } from "../src/lib/whiteboard/shapeRecognition";

const NL = String.fromCharCode(10);
let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`${(err as Error).message}`);
  }
}

type Pt = [number, number, number];

// Deterministic randomness, so a failure can be reproduced.
let seed = Number(process.env.SHAPE_SEED) || 12345;
const rand = () => {
  seed = (seed * 1103515245 + 12345) & 0x7fffffff;
  return seed / 0x7fffffff;
};

/** A path through `pts` sampled like a pen: ~3px apart, wobble + jitter. */
function handDrawn(path: Array<[number, number]>, wobble: number, jitter = 0.6): Pt[] {
  const out: Pt[] = [];
  const phase = rand() * 10;
  const freq = 0.02 + rand() * 0.02;
  let s = 0;
  for (let i = 0; i < path.length - 1; i++) {
    const [x0, y0] = path[i];
    const [x1, y1] = path[i + 1];
    const len = Math.hypot(x1 - x0, y1 - y0);
    const n = Math.max(1, Math.round(len / 3));
    const nx = -(y1 - y0) / (len || 1);
    const ny = (x1 - x0) / (len || 1);
    for (let k = 0; k < n; k++) {
      const t = k / n;
      s += len / n;
      const w = Math.sin(s * freq + phase) * wobble;
      out.push([
        x0 + (x1 - x0) * t + nx * w + (rand() - 0.5) * jitter,
        y0 + (y1 - y0) * t + ny * w + (rand() - 0.5) * jitter,
        0.45 + rand() * 0.1,
      ]);
    }
  }
  const [lx, ly] = path[path.length - 1];
  out.push([lx, ly, 0.5]);
  return out;
}

/** A closed outline from `corners` (or a dense curve), started at a random place, closed with a gap/overshoot
  . */
function closedStroke(outline: Array<[number, number]>, wobble: number, closeBy: number): Pt[] {
  const n = outline.length;
  const start = Math.floor(rand() * n);
  const loop: Array<[number, number]> = [];
  for (let i = 0; i <= n; i++) loop.push(outline[(start + i) % n]);
  // closeBy > 0: overshoot that fraction of the next side; < 0: stop short.
  if (closeBy > 0) {
    const a = loop[loop.length - 1];

```

```

    const b = outline[(start + 1) % n];
    loop.push([a[0] + (b[0] - a[0]) * closeBy, a[1] + (b[1] - a[1]) * closeBy]);
  } else if (closeBy < 0) {
    const a = loop[loop.length - 2];
    const b = loop[loop.length - 1];
    loop[loop.length - 1] = [b[0] + (a[0] - b[0]) * -closeBy, b[1] + (a[1] - b[1]) * -closeBy];
  }
  return handDrawn(loop, wobble);
}

function rotateAll(pts: Array<[number, number]>, ang: number, cx: number, cy: number): Array<[number, number]>
{
  return pts.map(([x, y]) => [
    cx + (x - cx) * Math.cos(ang) - (y - cy) * Math.sin(ang),
    cy + (x - cx) * Math.sin(ang) + (y - cy) * Math.cos(ang),
  ]);
}

function circleOutline(cx: number, cy: number, rx: number, ry: number, rot = 0, n = 72): Array<[number, number]>
{
  const out: Array<[number, number]> = [];
  for (let i = 0; i < n; i++) {
    const t = (i / n) * Math.PI * 2;
    out.push([cx + rx * Math.cos(t), cy + ry * Math.sin(t)]);
  }
  return rot ? rotateAll(out, rot, cx, cy) : out;
}

function bbox(pts: Array<[number, number] | Pt>) {
  const xs = pts.map((p) => p[0]);
  const ys = pts.map((p) => p[1]);
  return { x0: Math.min(...xs), y0: Math.min(...ys), x1: Math.max(...xs), y1: Math.max(...ys) };
}

/** Recognise `trials` drawings; each must be `want` and fit the ideal box within `tol` px. */
function expectShape(
  want: ShapeKind | ShapeKind[],
  make: () => { stroke: Pt[]; ideal: Array<[number, number]> },
  trials = 40,
  tol = 8
) {
  const wants = Array.isArray(want) ? want : [want];
  let wrong = 0;
  const kinds: Record<string, number> = {};
  for (let i = 0; i < trials; i++) {
    const { stroke, ideal } = make();
    const r = recognizeShape(stroke, { minSize: 20 });
    const k = r ? r.kind : "none";
    kinds[k] = (kinds[k] ?? 0) + 1;
    if (!r || !wants.includes(r.kind)) {
      wrong++;
      continue;
    }
    const a = bbox(r.points);
    const b = bbox(ideal);
    const off = Math.max(Math.abs(a.x0 - b.x0), Math.abs(a.y0 - b.y0), Math.abs(a.x1 - b.x1), Math.abs(a.y1 - b.y1));
    if (off > tol) {
      wrong++;
      kinds[`$${k}-misplaced(${off.toFixed(1)})`] = (kinds[`$${k}-misplaced(${off.toFixed(1)})`] ?? 0) + 1;
    }
  }
  // A hand is not a machine: 1 miss in 20 is allowed, never a wrong shape
  // counted as right.
  assert.ok(wrong <= Math.floor(trials * 0.05), `${wrong}/${trials} wrong: ${JSON.stringify(kinds)}`);
}

/** Must be left as drawn (null) at least this often. */
function expectNone(make: () => Pt[], trials = 40) {
  const kinds: Record<string, number> = {};
  let converted = 0;
  for (let i = 0; i < trials; i++) {
    const r = recognizeShape(make(), { minSize: 20 });
    if (r) {
      converted++;
      kinds[r.kind] = (kinds[r.kind] ?? 0) + 1;
    }
  }
}

```

```

    assert.ok(converted <= Math.floor(trials * 0.05), `${converted}/${trials} converted: ${JSON.stringify(kinds)}`);
  });
}

console.log(NL + "■■■ Shape recognition ■■■");

const R = (a: number, b: number) => a + rand() * (b - a);

test("circles become circles, same place and size", () => {
  expectShape("circle", () => {
    const r = R(30, 160);
    const cx = R(0, 500);
    const cy = R(0, 500);
    const ideal = circleOutline(cx, cy, r, r);
    return { stroke: closedStroke(ideal, r * R(0.01, 0.05), R(-0.12, 0.25) * 0.2), ideal };
  }, 60, 0.12 * 160);
});

test("ellipses become ellipses, upright or turned", () => {
  expectShape("ellipse", () => {
    const rx = R(80, 180);
    const ry = rx * R(0.4, 0.7);
    const cx = R(0, 500);
    const cy = R(0, 500);
    const ideal = circleOutline(cx, cy, rx, ry, rand() < 0.5 ? 0 : R(0.3, 1.2));
    return { stroke: closedStroke(ideal, ry * R(0.01, 0.04), R(-0.1, 0.2) * 0.2), ideal };
  }, 60, 16);
});

test("squares become squares", () => {
  expectShape("square", () => {
    const s = R(60, 240);
    const x = R(0, 400);
    const y = R(0, 400);
    const ideal: Array<[number, number]> = [[x, y], [x + s, y], [x + s, y + s], [x, y + s]];
    return { stroke: closedStroke(ideal, s * R(0.004, 0.015), R(-0.15, 0.2)), ideal };
  }, 60, 10);
});

test("rectangles become rectangles, also turned", () => {
  expectShape("rectangle", () => {
    const w = R(120, 300);
    const h = w * R(0.35, 0.7);
    const x = R(0, 400);
    const y = R(0, 400);
    let ideal: Array<[number, number]> = [[x, y], [x + w, y], [x + w, y + h], [x, y + h]];
    if (rand() < 0.5) ideal = rotateAll(ideal, R(0.2, 1.2), x + w / 2, y + h / 2);
    return { stroke: closedStroke(ideal, h * R(0.004, 0.015), R(-0.15, 0.2)), ideal };
  }, 60, 12);
});

test("triangles become triangles", () => {
  expectShape("triangle", () => {
    const s = R(80, 240);
    const x = R(0, 400);
    const y = R(0, 400);
    let ideal: Array<[number, number]> = [[x, y + s], [x + s, y + s], [x + s * R(0.2, 0.8), y + s * R(0, 0.2)]];
    if (rand() < 0.4) ideal = rotateAll(ideal, R(0.3, 2.5), x + s / 2, y + s / 2);
    return { stroke: closedStroke(ideal, s * R(0.004, 0.012), R(-0.12, 0.15)), ideal };
  }, 60, 12);
});

test("straight lines become ruler-straight, level when nearly level", () => {
  expectShape("line", () => {
    const len = R(60, 500);
    const ang = rand() < 0.5 ? R(-0.05, 0.05) : R(0, Math.PI * 2);
    const x = R(0, 400);
    const y = R(0, 400);
    const drawn: Array<[number, number]> = [[x, y], [x + Math.cos(ang) * len, y + Math.sin(ang) * len]];
    // Within 4 degrees of level or upright it is straightened, like a ruler.
    const q = Math.round(ang / (Math.PI / 2)) * (Math.PI / 2);
    const a = Math.abs(ang - q) <= (4 * Math.PI) / 180 ? q : ang;
    const ideal: Array<[number, number]> = [[x, y], [x + Math.cos(a) * len, y + Math.sin(a) * len]];
    return { stroke: handDrawn(drawn, R(0.5, 2)), ideal };
  }, 60, 8);
  // Nearly level comes out exactly level.
  const r = recognizeShape(handDrawn([[0, 100], [300, 108]], 1), { minSize: 20 });

```

```

    assert.equal(r.kind, "line");
    assert.ok(Math.abs(r.points[0][1] - r.points[r.points.length - 1][1]) < 1e-6, "level");
  });

test("pentagons and hexagons keep their corners", () => {
  for (const sides of [5, 6] as const) {
    expectShape(sides === 5 ? "pentagon" : "hexagon", () => {
      const r = R(90, 200);
      const cx = R(0, 400);
      const cy = R(0, 400);
      const off = rand() * Math.PI;
      const ideal: Array<number, number> = [];
      for (let i = 0; i < sides; i++) ideal.push([cx + r * Math.cos(off + (i / sides) * Math.PI * 2), cy + r *
        Math.sin(off + (i / sides) * Math.PI * 2)]);
      return { stroke: closedStroke(ideal, r * 0.006, R(-0.1, 0.1)), ideal };
    }, 30, 14);
  }
});

test("handwriting, arcs, zig-zags and scribbles are left as drawn", () => {
  // Cursive-like loops along a line.
  expectNone(() => {
    const pts: Array<number, number> = [];
    const x0 = R(0, 300);
    for (let t = 0; t < 200; t++) pts.push([x0 + t * 1.5 + Math.cos(t / 5) * 10, 100 + Math.sin(t / 5) * 18]);
    return handDrawn(pts, 0.5);
  });
  // An open arc (a "C").
  expectNone(() => {
    const pts: Array<number, number> = [];
    const r = R(40, 120);
    const span = R(Math.PI * 0.5, Math.PI * 1.3);
    for (let i = 0; i <= 60; i++) pts.push([200 + r * Math.cos(0.3 + (i / 60) * span), 200 + r * Math.sin(0.3
      + (i / 60) * span)]);
    return handDrawn(pts, r * 0.01);
  });
  // A zig-zag.
  expectNone(() => {
    const pts: Array<number, number> = [];
    for (let i = 0; i < 8; i++) pts.push([i * R(20, 40), i % 2 ? 0 : R(30, 60)]);
    return handDrawn(pts, 0.5);
  });
  // A dense scribble back and forth.
  expectNone(() => {
    const pts: Array<number, number> = [];
    for (let i = 0; i < 12; i++) pts.push([100 + (i % 2 ? 80 : 0) + R(-5, 5), 100 + i * 3]);
    return handDrawn(pts, 0.5);
  });
  // An "L" (two sides only).
  expectNone(() => handDrawn([[0, 0], [0, R(80, 200)], [R(80, 200), R(80, 200)], 0.8]));
});

test("rounded corners and sloppy closing still read as the shape", () => {
  // A square drawn with rounded corners (how most hands draw one).
  expectShape("square", () => {
    const s = R(80, 220);
    const x = R(0, 400);
    const y = R(0, 400);
    const rr = s * R(0.05, 0.12);
    const out: Array<number, number> = [];
    const corners: Array<number, number, number> = [
      [x + s - rr, y + rr, -Math.PI / 2],
      [x + s - rr, y + s - rr, 0],
      [x + rr, y + s - rr, Math.PI / 2],
      [x + rr, y + rr, Math.PI],
    ];
    for (const [cx, cy, a0] of corners) for (let k = 0; k <= 6; k++) out.push([cx + rr * Math.cos(a0 + (k / 6)
      * (Math.PI / 2)), cy + rr * Math.sin(a0 + (k / 6) * (Math.PI / 2))]);
    const ideal: Array<number, number> = [[x, y], [x + s, y], [x + s, y + s], [x, y + s]];
    return { stroke: closedStroke(out, s * 0.008, R(-0.05, 0.1)), ideal };
  }, 40, 12);
  // A wobblier circle, closed with a larger gap.
  expectShape("circle", () => {
    const r = R(50, 150);
    const ideal = circleOutline(200, 200, r, r);
    return { stroke: closedStroke(ideal, r * 0.06, -0.1), ideal };
  }, 40, 0.15 * 150);
});

```

```

test("a star, a heart, a figure-8, a spiral and a letter D are left as drawn", () => {
  expectNone(() => {
    const pts: Array<[number, number]> = [];
    const r = R(60, 150);
    for (let i = 0; i < 10; i++) {
      const rad = i % 2 ? r * 0.4 : r;
      pts.push([200 + rad * Math.cos(-Math.PI / 2 + (i / 10) * Math.PI * 2), 200 + rad * Math.sin(-Math.PI / 2
        + (i / 10) * Math.PI * 2)]);
    }
    return closedStroke(pts, r * 0.005, 0.02);
  }, 30);
  expectNone(() => {
    const pts: Array<[number, number]> = [];
    const k = R(4, 7);
    for (let i = 0; i < 80; i++) {
      const t = (i / 80) * Math.PI * 2;
      pts.push([200 + k * 16 * Math.sin(t) ** 3, 200 - k * (13 * Math.cos(t) - 5 * Math.cos(2 * t) - 2 * Math.
        cos(3 * t) - Math.cos(4 * t))]);
    }
    return closedStroke(pts, 1, 0.02);
  }, 30);
  expectNone(() => {
    const pts: Array<[number, number]> = [];
    const r = R(40, 90);
    for (let i = 0; i < 80; i++) {
      const t = (i / 80) * Math.PI * 2;
      pts.push([200 + r * Math.sin(t), 200 + r * Math.sin(t) * Math.cos(t) * 2]);
    }
    return closedStroke(pts, 0.5, 0.02);
  }, 30);
  expectNone(() => {
    const pts: Array<[number, number]> = [];
    for (let i = 0; i < 120; i++) {
      const t = (i / 120) * Math.PI * 5;
      pts.push([200 + t * 8 * Math.cos(t), 200 + t * 8 * Math.sin(t)]);
    }
    return handDrawn(pts, 0.5);
  }, 30);
  expectNone(() => {
    const h = R(80, 160);
    const pts: Array<[number, number]> = [[100, 100], [100, 100 + h]];
    for (let i = 0; i <= 20; i++) {
      const t = Math.PI / 2 - (i / 20) * Math.PI;
      pts.push([100 + (h / 2) * 1.1 * Math.cos(t), 100 + h / 2 + (h / 2) * Math.sin(-t)]);
    }
    return handDrawn(pts, 0.6);
  }, 30);
});

test("tiny marks are never shapes (a dot, a letter o)", () => {
  assert.equal(recognizeShape(closedStroke(circleOutline(10, 10, 6, 6), 0.2, 0), { minSize: 20 }), null);
});

test("output stays in the stroke's own coordinates and is closed", () => {
  const ideal = circleOutline(300, 200, 80, 80);
  const r = recognizeShape(closedStroke(ideal, 1, 0.02), { minSize: 20 })!;
  assert.equal(r.kind, "circle");
  const a = r.points[0];
  const b = r.points[r.points.length - 1];
  assert.ok(Math.hypot(a[0] - b[0], a[1] - b[1]) < 1e-6, "ends meet");
  assert.ok(r.points.every((p) => p[2] > 0 && p[2] <= 1), "pressure kept sensible");
});

test("real strokes drawn by hand: every shape meant is found, the rest left alone", () => {
  const file = path.join(process.cwd(), "tests", "fixtures", "shape_user_strokes.json");
  const { strokes } = JSON.parse(fs.readFileSync(file, "utf8")) as {
    strokes: Array<{ id: string; expect: string; points: Pt[] }>;
  };
  const wrong: string[] = [];
  for (const s of strokes) {
    const r = recognizeShape(s.points, { minSize: 28 });
    const got = r ? r.kind : "none";
    if (!s.expect.split("|").includes(got)) wrong.push(`${s.id}: ${got} (want ${s.expect})`);
  }
  assert.deepEqual(wrong, []);
});

```

```

test("an overshoot, a tail past the start or a crossing still closes the shape", () => {
  // A square whose pen runs on a quarter side past where it began.
  const sq: Array<number, number> = [[0, 0], [200, 0], [200, 200], [0, 200], [0, 0], [120, 0]];
  assert.equal(recognizeShape(handDrawn(sq, 1), { minSize: 20 })?.kind, "square");
  // A circle that crosses over its start and carries on.
  const c = circleOutline(200, 200, 90, 90);
  const loop = [...c, ...c.slice(0, 14)];
  assert.equal(recognizeShape(handDrawn(loop, 1.5), { minSize: 20 })?.kind, "circle");
});

test("lines drawn one after another join at clean corners", () => {
  // The first line ran a little past the corner; the second starts short of it.
  const first = { id: "a", line: [[0, 0], [212, 3]] as [[number, number], [number, number]] };
  const j = joinLine([195, 14], [201, 180], [first], 30);
  const corner = j.start;
  assert.ok(Math.abs(corner[1] - (corner[0] * 3) / 212) < 0.5, "the corner is on the first line");
  assert.equal(j.adjust.length, 1, "the first line is trimmed to the corner");
  assert.deepEqual(j.adjust[0].to, corner);
  assert.equal(j.adjust[0].end, 1);
  // Far away: nothing moves.
  const far = joinLine([400, 400], [500, 500], [first], 30);
  assert.deepEqual([far.start, far.end, far.adjust.length], [[400, 400], [500, 500], 0]);
  // Near a triangle's corner: the end lands on it.
  const tri = { id: "t", corners: [[50, 50], [150, 50], [100, 130]] as [number, number][] };
  assert.deepEqual(joinLine([148, 56], [300, 60], [tri], 20).start, [150, 50]);
});

console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/shape_recognition.test.ts ===
=====

=====
// === START FILE: tests/shape_tools.test.ts ===
// File 179 of 194 | Lines: 148 | Size: 6024 bytes
=====

/**
 * Tests for the shape-tool rework:
 * (A) the quick color picker colors the fill too, not just the edges;
 * (D) a tap (no drag) plants a default-size shape instead of nothing;
 * (E) a fingertip tap selects at once instead of needing a long press;
 * (P) the inspector folds to its header (persisted) so it never buries
 * the canvas; inserting returns to select, so only one panel is ever open.
 */

import assert from "node:assert/strict";
import { DEFAULT_STYLE, setToolInk, type ToolType } from "../src/lib/whiteboard/types";
import { TAP_SHAPE_WIDTH, TAP_SHAPE_HEIGHT, tapShapeRect } from "../src/lib/whiteboard/placement";
import { shouldSelectOnTouchTap } from "../src/lib/whiteboard/palm";
import { useWhiteboard } from "../src/store/whiteboard";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(` ${(err as Error).message}`);
  }
}

const SHAPES: ToolType[] = ["rectangle", "ellipse", "diamond", "triangle", "star", "cloud"];

console.log(`\n■■ Shape tools: fill, tap-insert, tap-select ■■`);

test("quick color pick on a shape tool colors the fill as well as the edge", () => {
  for (const tool of SHAPES) {
    const patch = setToolInk(DEFAULT_STYLE, tool, { color: "#ff0000" });
    assert.equal(
      (patch as { shapeStrokeColor?: string }).shapeStrokeColor,

```

```

    "#ff0000",
    `${tool}: edge must follow the pick`
  );
  assert.equal(
    (patch as { shapeFillColor?: string }).shapeFillColor,
    "#ff0000",
    `${tool}: fill must follow the pick (was stuck default blue)`
  );
}
});

test("quick color pick on other tools does not touch shape fill", () => {
  const line = setToolInk(DEFAULT_STYLE, "line", { color: "#00ff00" });
  assert.equal((line as { lineColor?: string }).lineColor, "#00ff00");
  assert.equal("shapeFillColor" in line, false, "line pick must not write shape fill");
  const text = setToolInk(DEFAULT_STYLE, "text", { color: "#0000ff" });
  assert.equal((text as { textColor?: string }).textColor, "#0000ff");
  assert.equal("shapeFillColor" in text, false, "text pick must not write shape fill");
  const pen = setToolInk(DEFAULT_STYLE, "gel", { color: "#123456", size: 5 });
  assert.equal("shapeFillColor" in pen, false, "pen pick must not write shape fill");
});

test("tap shape is centred on the tap point at the default size", () => {
  const r = tapShapeRect(400, 300);
  assert.equal(r.width, TAP_SHAPE_WIDTH);
  assert.equal(r.height, TAP_SHAPE_HEIGHT);
  assert.equal(r.x, 400 - TAP_SHAPE_WIDTH / 2);
  assert.equal(r.y, 300 - TAP_SHAPE_HEIGHT / 2);
});

test("tap shape is capped to the sheet and translated fully onto it", () => {
  const page = { width: 200, height: 200 };
  // A 160x120 default fits a 200x200 sheet as-is, centred.
  const mid = tapShapeRect(100, 100, { page });
  assert.equal(mid.width, TAP_SHAPE_WIDTH);
  assert.equal(mid.x, 100 - TAP_SHAPE_WIDTH / 2);
  assert.ok(mid.x >= 0 && mid.x + mid.width <= 200, "must sit fully on the sheet");
  assert.ok(mid.y >= 0 && mid.y + mid.height <= 200, "must sit fully on the sheet");

  // A tap at the corner pulls the whole shape on paper, never off it.
  const corner = tapShapeRect(5, 5, { page });
  assert.equal(corner.x, 0);
  assert.equal(corner.y, 0);

  // A sheet smaller than the default is never exceeded.
  const tiny = tapShapeRect(50, 50, { page: { width: 100, height: 60 } });
  assert.ok(tiny.width <= 100 && tiny.height <= 60, "must never be born bigger than the page");
  assert.ok(tiny.x >= 0 && tiny.y >= 0, "must stay on a tiny sheet");
});

test("tap shape is snapped when grid snap is on, untouched otherwise", () => {
  const snap = (v: number) => Math.round(v / 10) * 10;
  const snapped = tapShapeRect(403, 307, { snap });
  assert.equal(snapped.x, Math.round((403 - TAP_SHAPE_WIDTH / 2) / 10) * 10);
  const plain = tapShapeRect(403, 307);
  assert.equal(plain.x, 403 - TAP_SHAPE_WIDTH / 2, "no snap fn means no rounding");
});

test("fingertip tap on a fresh object selects at once", () => {
  assert.equal(
    shouldSelectOnTouchTap({ verdict: "allow", hitSelected: false, shiftKey: false, editing: false }),
    true,
    "clean tap must select immediately"
  );
});

test("tap-select stays off for palms, selections, shift and mid-edit", () => {
  assert.equal(
    shouldSelectOnTouchTap({ verdict: "contact-size", hitSelected: false, shiftKey: false, editing: false }),
    false,
    "a palm-sized patch still needs the long press"
  );
  assert.equal(
    shouldSelectOnTouchTap({ verdict: "pen-down", hitSelected: false, shiftKey: false, editing: false }),
    false,
    "pen-suppressed contact must not select"
  );
  assert.equal(

```

```

    shouldSelectOnTouchTap({ verdict: "allow", hitSelected: true, shiftKey: false, editing: false })),
    false,
    "already-selected keeps its own (move/menu) path"
  );
  assert.equal(
    shouldSelectOnTouchTap({ verdict: "allow", hitSelected: false, shiftKey: true, editing: false })),
    false,
    "shift keeps toggling"
  );
  assert.equal(
    shouldSelectOnTouchTap({ verdict: "allow", hitSelected: false, shiftKey: false, editing: true })),
    false,
    "mid-edit touch stays a resting hand"
  );
});

test("inspector fold state persists through the store", () => {
  const S = () => useWhiteboard.getState();
  assert.equal(S().propsPanelCollapsed, true, "inspector starts folded so it never buries the canvas");
  S().setPropsPanelCollapsed(false);
  assert.equal(S().propsPanelCollapsed, false, "unfolding flips the flag");
  assert.equal(
    S().getPersistedState().propsPanelCollapsed,
    false,
    "the flag must survive a reload like every other panel toggle"
  );
  S().setPropsPanelCollapsed(true);
  assert.equal(S().getPersistedState().propsPanelCollapsed, true, "folding persists too");
});

console.log(`\n${passed} passed, ${failed} failed.`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/shape_tools.test.ts ===
=====

=====
// === START FILE: tests/share_insert.test.ts ===
// File 180 of 194 | Lines: 497 | Size: 18482 bytes
=====

/**
 * Exporting a page range, and inserting pages into a notebook.
 *
 * Two features that meet in the same place: a notebook is no longer all-or-
 * nothing. You can hand in pages 4 to 9, and you can pull pages 2 to 5 of last
 * year's notes (or of a PDF) into the middle of this one.
 *
 * The things most likely to go wrong, and so the things pinned here:
 * - a range typed backwards, past the end, or half-typed must never export
 *   nothing or throw - it must mean the obvious thing;
 * - copied pages must get FRESH element ids, or selecting one selects both
 *   copies and deleting one deletes both;
 * - a connector whose other end was left behind must be dropped, not carried
 *   over pointing at an element that does not exist;
 * - inserting must COPY, never move - the source notebook is someone's work;
 * - a copied PDF page must keep the whole notebook's record of its documents
 *   straight, or the storage collector deletes a PDF still being read.
 */

import assert from "node:assert/strict";

import { useWhiteboard } from "../src/store/whiteboard";
import {
  getPages,
  allElements,
  resolvePageRange,
  parsePageSelection,
  formatPageSelection,
  copyPagesFrom,
  insertPages,
  pageFromPdf,
} from "../src/lib/whiteboard/notebook";
import { buildExportSvg, exportFilename } from "../src/lib/whiteboard/export";
import { isShareCancellation, shareRoute, canChooseApp } from "../src/lib/whiteboard/share";
import type {

```

```

Board,
NotebookPage,
StrokeElement,
ConnectionElement,
WhiteboardElement,
} from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`${(err as Error).message}`);
  }
}

const S = () => useWhiteboard.getState();
const board = (id: string): Board => S().boards.find((b) => b.id === id)!;

let seq = 0;
function stroke(extra: Partial<StrokeElement> = {}): StrokeElement {
  seq++;
  return {
    id: `s${seq}`,
    type: "stroke",
    x: 10,
    y: 10,
    width: 50,
    height: 50,
    rotation: 0,
    opacity: 1,
    // Points are [x, y, pressure] tuples, not objects.
    points: [
      [0, 0, 0.5],
      [10, 10, 0.5],
      [20, 18, 0.5],
    ],
    color: "#000000",
    strokeWidth: 2,
    penType: "pen",
    createdAt: 1,
    updatedAt: 1,
    ...extra,
  } as StrokeElement;
}

function connection(id: string, fromId: string, toId: string, boardId: string): ConnectionElement {
  return {
    id,
    type: "connection",
    x: 0,
    y: 0,
    width: 0,
    height: 0,
    rotation: 0,
    opacity: 1,
    source: { boardId, elementId: fromId },
    target: { boardId, elementId: toId },
    createdAt: 1,
    updatedAt: 1,
  } as unknown as ConnectionElement;
}

function page(elements: WhiteboardElement[] = [], extra: Partial<NotebookPage> = {}): NotebookPage {
  seq++;
  return { id: `pg${seq}`, elements, createdAt: 1, updatedAt: 1, ...extra };
}

/** A detached notebook, not in the store - for the pure helpers. */
function notebook(pages: NotebookPage[], id = "src"): Board {
  return {
    id,
    name: "Source",
  }

```

```

    folderId: null,
    elements: [],
    camera: { x: 0, y: 0, zoom: 1 },
    kind: "notebook",
    pages,
    activePageId: pages[0]?.id,
    createdAt: 1,
    updatedAt: 1,
  } as Board;
}

/** A real notebook in the store, with `count` pages each holding one stroke. */
function storeNotebook(name: string, count: number): string {
  const id = S().createNotebook(name, null);
  S().setActiveBoard(id);
  for (let i = 0; i < count; i++) {
    if (i > 0) S().addPage();
    S().addElement(stroke(), true);
  }
  return id;
}

console.log("\n■■■ Page ranges: what a typed range means ■■■");

test("a plain range becomes the pages between its ends, inclusive", () => {
  const r = resolvePageRange(10, 2, 5);
  assert.deepEqual(r.indices, [1, 2, 3, 4]);
  assert.equal(r.from, 2);
  assert.equal(r.to, 5);
});

test("a single page is a range of one", () => {
  assert.deepEqual(resolvePageRange(10, 3, 3).indices, [2]);
});

test("a backwards range means what it says, not nothing", () => {
  // Typing "5 to 2" is a person describing pages 2 to 5 in the wrong order.
  assert.deepEqual(resolvePageRange(10, 5, 2).indices, resolvePageRange(10, 2, 5).indices);
});

test("a range past the end stops at the last page", () => {
  const r = resolvePageRange(10, 8, 999);
  assert.deepEqual(r.indices, [7, 8, 9]);
  assert.equal(r.to, 10);
});

test("zero and negatives clamp to the first page", () => {
  assert.deepEqual(resolvePageRange(5, 0, 2).indices, [0, 1]);
  assert.deepEqual(resolvePageRange(5, -7, 1).indices, [0]);
});

test("a half-typed range is still a range", () => {
  // The box is empty while you type, which arrives as NaN. It must not throw
  // and must not silently select nothing.
  assert.deepEqual(resolvePageRange(4, NaN, NaN).indices, [0, 1, 2, 3]);
  assert.deepEqual(resolvePageRange(4, 2, NaN).indices, [1, 2, 3]);
});

test("a notebook with no pages yields no pages", () => {
  assert.deepEqual(resolvePageRange(0, 1, 5).indices, []);
});

console.log("\n-- Typing a page selection --");

test("the shorthand every note app accepts", () => {
  assert.deepEqual(parsePageSelection("2-5, 8", 10), [2, 3, 4, 5, 8]);
  assert.deepEqual(parsePageSelection("1,3", 10), [1, 3]);
  assert.deepEqual(parsePageSelection("7", 10), [7]);
});

test("open-ended ranges mean from-the-start and to-the-end", () => {
  assert.deepEqual(parsePageSelection("8-", 10), [8, 9, 10]);
  assert.deepEqual(parsePageSelection("-3", 10), [1, 2, 3]);
});

test("messy input still means something sensible", () => {
  assert.deepEqual(parsePageSelection(" 4 - 2 ", 10), [2, 3, 4], "backwards and spaced");
  assert.deepEqual(parsePageSelection("2-4,", 10), [2, 3, 4], "trailing comma");
});

```

```

    assert.deepEqual(parsePageSelection("2\u2013", 10), [2, 3, 4, 5], "en-dash pasted from a PDF");
    assert.deepEqual(parsePageSelection("3,3,3", 10), [3], "duplicates collapse");
    assert.deepEqual(parsePageSelection("9-99", 10), [9, 10], "clamped to what exists");
  });

  test("nothing usable selects nothing, rather than throwing", () => {
    assert.deepEqual(parsePageSelection("", 10), []);
    assert.deepEqual(parsePageSelection("abc", 10), []);
    assert.deepEqual(parsePageSelection("-", 10), []);
    assert.deepEqual(parsePageSelection("1-3", 0), []);
  });

  test("a selection reads back the way it was typed", () => {
    assert.equal(formatPageSelection([1, 2, 3, 5]), "1-3, 5");
    assert.equal(formatPageSelection([4]), "4");
    assert.equal(formatPageSelection([]), "");
    assert.equal(formatPageSelection([3, 1, 2]), "1-3", "order does not matter");
  });

  test("typing a range and ticking pages are exact inverses", () => {
    // The dialog keeps the range box and the thumbnails in sync; that only
    // holds if these two round-trip.
    assert.equal(formatPageSelection(parsePageSelection("2-5, 8", 10)), "2-5, 8");
  });

  console.log("\n■■■ Exporting a page range ■■■");

  test("the export contains exactly the pages asked for", () => {
    const nb = notebook([page([stroke()]), page([stroke()]), page([stroke()]), page([stroke()])]);
    assert.equal(buildExportSvg(nb, { pages: { from: 2, to: 3 } }).report.pages, 2);
    assert.equal(buildExportSvg(nb, { pages: "all" }).report.pages, 4);
    assert.equal(buildExportSvg(nb, { pages: "current" }).report.pages, 1);
  });

  test("the range picks the RIGHT pages, not just the right number", () => {
    // Each page carries an identifiable piece of text, so the export can be
    // searched for it. Counting sheets alone would pass even if the export
    // always took the first N pages.
    const mk = (t: string) =>
      page([
        {
          id: `t_${t}`,
          type: "text",
          x: 5,
          y: 5,
          width: 100,
          height: 20,
          rotation: 0,
          opacity: 1,
          text: t,
          color: "#000",
          fontSize: 12,
          fontFamily: "Inter",
          fontWeight: 400,
          align: "left",
          createdAt: 1,
          updatedAt: 1,
        }
      ]) as unknown as WhiteboardElement,
    []);
    const nb = notebook([mk("ALPHA"), mk("BRAVO"), mk("CHARLIE"), mk("DELTA")]);
    const svg = buildExportSvg(nb, { pages: { from: 2, to: 3 } }).svg;
    assert.ok(!svg.includes("ALPHA"), "page 1 was left out");
    assert.ok(svg.includes("BRAVO"), "page 2 is in");
    assert.ok(svg.includes("CHARLIE"), "page 3 is in");
    assert.ok(!svg.includes("DELTA"), "page 4 was left out");
  });

  test("a backwards range exports rather than failing", () => {
    const nb = notebook([page(), page(), page()]);
    assert.equal(buildExportSvg(nb, { pages: { from: 3, to: 1 } }).report.pages, 3);
  });

  test("the filename says which pages are inside", () => {
    const nb = notebook([page(), page(), page(), page()]);
    nb.name = "Calculus Notes";
    assert.equal(exportFilename(nb, "pdf", { pages: { from: 2, to: 4 } }), "Calculus_Notes_p2-4.pdf");
    assert.equal(exportFilename(nb, "png", { pages: { from: 3, to: 3 } }), "Calculus_Notes_p3.png");
    assert.equal(exportFilename(nb, "pdf", { pages: "all" }), "Calculus_Notes.pdf");
  });

```

```

});

test("a name full of punctuation still produces a legal filename", () => {
  const nb = notebook([page()]);
  nb.name = 'Week 3: "derivatives"/limits*?';
  const name = exportFilename(nb, "pdf", {});
  assert.ok(!/[:"/*?]/.test(name), `still has punctuation: ${name}`);
  assert.ok(name.endsWith(".pdf"));
});

test("every format gets its own extension", () => {
  const nb = notebook([page()]);
  nb.name = "n";
  assert.ok(exportFilename(nb, "jpeg", {}).endsWith(".jpg"), "jpeg is .jpg, as the world writes it");
  assert.ok(exportFilename(nb, "webp", {}).endsWith(".webp"));
  assert.ok(exportFilename(nb, "svg", {}).endsWith(".svg"));
});

console.log("\n■■■ Copying pages between notebooks ■■■");

test("copied pages are independent of the originals", () => {
  const a = stroke();
  const src = notebook([page([a]), page([stroke()])]);
  const copies = copyPagesFrom(src, [0], "target", () => `new${seq++}`);

  assert.equal(copies.length, 1);
  assert.notEqual(copies[0].id, src.pages![0].id, "the page has a new id");
  assert.notEqual(copies[0].elements[0].id, a.id, "the element has a new id");
  assert.equal(src.pages![0].elements[0].id, a.id, "the source is untouched");
});

test("a connector between two copied elements follows them", () => {
  const a = stroke();
  const b = stroke();
  const conn = connection("c1", a.id, b.id, "src");
  const src = notebook([page([a, b, conn])]);

  const copies = copyPagesFrom(src, [0], "target", () => `new${seq++}`);
  const copied = copies[0].elements.find((e) => e.type === "connection") as ConnectionElement;

  assert.ok(copied, "the connector came along");
  assert.equal(copied.source.boardId, "target", "it points at the new board");
  assert.notEqual(copied.source.elementId, a.id, "at the new element ids");
  const ids = copies[0].elements.map((e) => e.id);
  assert.ok(ids.includes(copied.source.elementId), "which exist on the copied page");
  assert.ok(ids.includes(copied.target.elementId));
});

test("a connector to a page left behind is dropped, not left dangling", () => {
  const a = stroke();
  const far = stroke();
  const conn = connection("c2", a.id, far.id, "src");
  const src = notebook([page([a, conn]), page([far])]);

  // Copy only page 1: `far` stays behind.
  const copies = copyPagesFrom(src, [0], "target", () => `new${seq++}`);
  assert.equal(
    copies[0].elements.some((e) => e.type === "connection"),
    false,
    "the half-attached connector was removed"
  );
  assert.equal(copies[0].elements.length, 1, "the stroke itself is still there");
});

test("a connector to a DIFFERENT board keeps pointing there", () => {
  const a = stroke();
  const conn = connection("c3", a.id, "elsewhere", "src");
  conn.target.boardId = "other-board";
  const src = notebook([page([a, conn])]);

  const copied = copyPagesFrom(src, [0], "target", () => `new${seq++}`)[0].elements.find(
    (e) => e.type === "connection"
  ) as ConnectionElement;
  assert.ok(copied, "a cross-board link survives the copy");
  assert.equal(copied.target.boardId, "other-board");
  assert.equal(copied.target.elementId, "elsewhere", "untouched - it was never ours to remap");
});

```

```

test("a PDF-backed page keeps its reference and its paper", () => {
  const src = notebook([pageFromPdf("doc1", 7)]);
  const copy = copyPagesFrom(src, [0], "target", () => "x")[0];
  assert.deepEqual(copy.pdf, { docId: "doc1", pageNumber: 7 });
  assert.equal(copy.background?.pattern, "plain", "a PDF page is the paper; no dots over a slide");
});

console.log("\n■■■ Inserting pages at a position ■■■");

test("pages land exactly where asked", () => {
  const nb = notebook([page(), page(), page()]);
  const mark = page();
  mark.id = "INSERTED";

  assert.equal(getPages(insertPages(nb, [mark], 0)).findIndex((p) => p.id === "INSERTED"), 0);
  assert.equal(getPages(insertPages(nb, [mark], 1)).findIndex((p) => p.id === "INSERTED"), 1);
  assert.equal(getPages(insertPages(nb, [mark], 3)).findIndex((p) => p.id === "INSERTED"), 3);
});

test("a position past the end appends instead of throwing away the pages", () => {
  const nb = notebook([page(), page()]);
  const mark = page();
  mark.id = "LAST";
  const out = insertPages(nb, [mark], 999);
  assert.equal(getPages(out).length, 3);
  assert.equal(getPages(out)[2].id, "LAST");
});

test("the first inserted page becomes the one you are looking at", () => {
  const nb = notebook([page(), page()]);
  const a = page();
  a.id = "FIRST_NEW";
  const b = page();
  const out = insertPages(nb, [a, b], 1);
  assert.equal(out.activePageId, "FIRST_NEW", "you are taken to what you just inserted");
});

test("inserting nothing changes nothing", () => {
  const nb = notebook([page(), page()]);
  assert.equal(insertPages(nb, [], 1), nb);
});

console.log("\n■■■ Inserting through the store ■■■");

test("a range of another notebook's pages is copied in, and the source is untouched", () => {
  const source = storeNotebook("Source NB", 5);
  const target = storeNotebook("Target NB", 2);

  const n = S().insertPagesFromBoard(source, { from: 2, to: 4 }, 1);

  assert.equal(n, 3, "three pages inserted");
  assert.equal(getPages(board(target)).length, 5, "2 + 3");
  assert.equal(getPages(board(source)).length, 5, "the source still has all of its pages");
});

test("the inserted pages carry the source's ink, under new ids", () => {
  const source = storeNotebook("Ink Source", 3);
  const sourceIds = allElements(board(source)).map((e) => e.id);
  const target = storeNotebook("Ink Target", 1);

  S().insertPagesFromBoard(source, { from: 1, to: 3 }, 1);

  const targetEls = allElements(board(target));
  assert.equal(targetEls.length, 4, "one of its own plus three copied strokes");
  for (const id of targetEls.map((e) => e.id)) {
    assert.ok(!sourceIds.includes(id), `element ${id} reused a source id`);
  }
});

test("pages land at the position given", () => {
  const source = storeNotebook("Pos Source", 2);
  const target = storeNotebook("Pos Target", 3);
  const before = getPages(board(target)).map((p) => p.id);

  S().insertPagesFromBoard(source, { from: 1, to: 1 }, 0);

  const after = getPages(board(target)).map((p) => p.id);
  assert.equal(after.length, 4);
});

```

```

    assert.deepEqual(after.slice(1), before, "the original pages all moved down by one");
  });

test("a whiteboard inserts as a single page", () => {
  const canvas = S().createBoard("A sketch", null);
  S().setActiveBoard(canvas);
  S().addElement(stroke(), true);
  const target = storeNotebook("Mixed NB", 1);

  const n = S().insertPagesFromBoard(canvas, { from: 1, to: 1 }, 1);
  assert.equal(n, 1, "the canvas counts as one page");
  assert.equal(getPages(board(target)).length, 2);
});

test("inserting into a whiteboard does nothing rather than corrupting it", () => {
  const source = storeNotebook("Src", 2);
  const canvas = S().createBoard("Not a notebook", null);
  S().setActiveBoard(canvas);

  assert.equal(S().insertPagesFromBoard(source, { from: 1, to: 2 }, 0), 0);
  assert.equal(getPages(board(canvas)).length, 0);
});

test("a copied PDF page is recorded on the new notebook, so it is not garbage-collected", () => {
  // The storage collector deletes any PDF no page references. A copied page
  // references it, and the target board must say so too or its own record of
  // its documents is a lie.
  const source = storeNotebook("PDF Source", 1);
  const pages = getPages(board(source));
  useWhiteboard.setState((s) => ({
    boards: s.boards.map((b) => {
      b.id === source
        ? { ...b, pages: [{ ...pages[0], pdf: { docId: "doc-xyz", pageNumber: 1 } }], pdfDocIds: ["doc-xyz"] }
        : b
    },
  }));

  const target = storeNotebook("PDF Target", 1);
  S().insertPagesFromBoard(source, { from: 1, to: 1 }, 1);

  const t = board(target);
  assert.deepEqual(getPages(t)[1].pdf, { docId: "doc-xyz", pageNumber: 1 }, "the page still shows the PDF");
  assert.ok(t.pdfDocIds?.includes("doc-xyz"), "and the notebook lists the document");
});

test("an out-of-range request inserts nothing instead of blank pages", () => {
  const source = storeNotebook("Small", 2);
  const target = storeNotebook("Dest", 1);
  // Pages 9-12 of a 2-page notebook: clamped to the last page, not invented.
  const n = S().insertPagesFromBoard(source, { from: 9, to: 12 }, 1);
  assert.equal(n, 1, "clamped to the one page that exists");
  assert.equal(getPages(board(target)).length, 2);
});

console.log("\n■■■ Sharing: where a file goes ■■■");

test("dismissing a share sheet is not an error", () => {
  assert.equal(isShareCancellation({ name: "AbortError" }), true, "the Web Share API's name for it");
  assert.equal(isShareCancellation(new Error("Share canceled")), true, "Android's wording");
  assert.equal(isShareCancellation(new Error("Share cancelled")), true);
  assert.equal(isShareCancellation(new Error("No space left on device")), false, "a real failure is not swallowed");
  assert.equal(isShareCancellation(null), false);
});

test("without a system share sheet, files fall back to a download", () => {
  // Node has no navigator.share, which is exactly the Electron case.
  assert.equal(canChooseApp(), false);
  assert.equal(shareRoute(), "download");
});

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/share_insert.test.ts ===
=====

```

```

=====
// === START FILE: tests/sidebar_batch.test.ts ===
// File 181 of 194 | Lines: 130 | Size: 3975 bytes
=====

/**
 * Sidebar pro work: batch duplicate + batch export packing.
 *
 * Duplicating several boards reuses the single duplicateBoard path (full
 * notebook copies included); packing several exports into one .zip reuses
 * the per-board SVG pipeline. Both are store/lib-level and node-testable.
 */

import assert from "node:assert/strict";
import { zipSync, unzipSync, strToU8, strFromU8 } from "fflate";
import { useWhiteboard } from "../src/store/whiteboard";
import {
  exportBoardToSvg,
  exportFilename,
  zipExportFiles,
} from "../src/lib/whiteboard/export";
import type { Board } from "../src/lib/whiteboard/types";

const NL = String.fromCharCode(10);

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`   ${(err as Error).message}`);
  }
}

function resetStore() {
  useWhiteboard.setState({
    boards: [],
    folders: [],
    activeBoardId: null,
    activeFolderId: null,
    selectedIds: [],
    editingId: null,
    history: { past: [], future: [] },
  });
  useWhiteboard.getState().discardTrash();
}

let seq = 0;
function board(partial: Partial<Board> = {}): Board {
  seq++;
  return {
    id: `d${seq}`,
    name: `Doc ${seq}`,
    folderId: null,
    elements: [],
    background: { pattern: "plain" },
    camera: { x: 0, y: 0, zoom: 1 },
    createdAt: seq,
    updatedAt: seq,
    ...partial,
  } as Board;
}

function main() {
  console.log(NL + "■■■ Sidebar batch ops ■■■");

  test("batchDuplicateBoards copies canvas boards with fresh ids", () => {
    resetStore();
    const S = () => useWhiteboard.getState();
    const a = board({ id: "a", name: "A" });
    useWhiteboard.setState({ boards: [a], activeBoardId: "a" });
    const ids = S().batchDuplicateBoards(["a", "missing"]);
  });
}

```

```

    assert.equal(ids.length, 1, "unknown ids are skipped");
    const copy = S().boards.find((b) => b.id === ids[0])!;
    assert.ok(copy, "copy exists");
    assert.notEqual(copy.id, "a");
    assert.equal(copy.name, "A (Copy)");
  });

  test("batchDuplicateBoards copies every notebook page", () => {
    resetStore();
    const S = () => useWhiteboard.getState();
    const nb = board({
      id: "nb",
      name: "NB",
      kind: "notebook",
      pages: [1, 2, 3].map((n) => ({
        id: `p${n}`,
        elements: [
          { id: `e${n}`, type: "text", x: 0, y: 0, rotation: 0, opacity: 1, text: `page ${n}`, createdAt: 1, u
            pdatedAt: 1 } as never,
        ],
        createdAt: 1,
        updatedAt: 1,
      })),
    });
    useWhiteboard.setState({ boards: [nb], activeBoardId: "nb" });
    const ids = S().batchDuplicateBoards(["nb"]);
    const copy = S().boards.find((b) => b.id === ids[0])!;
    assert.equal(copy.pages?.length, 3, "all pages copied");
    assert.equal(copy.pages?.[0]?.elements.length, 1);
    assert.notEqual(copy.pages?.[0]?.elements[0]?.id, "e1", "elements get fresh ids");
  });

  test("fflate itself round-trips (library sanity for the zip path)", () => {
    const zipped = zipSync({ "a.svg": strToU8("<svg/>") });
    const back = unzipSync(zipped);
    assert.equal(strFromU8(back["a.svg"]), "<svg/>");
  });

  test("two board SVGs pack into one zip under their export filenames", () => {
    const b1 = board({ id: "z1", name: "First" });
    const b2 = board({ id: "z2", name: "Second" });
    const files = [b1, b2].map((b) => ({
      filename: exportFilename(b, "svg"),
      data: strToU8(exportBoardToSvg(b)),
    }));
    assert.deepEqual(
      files.map((f) => f.filename),
      ["First.svg", "Second.svg"]
    );
    const back = unzipSync(zipExportFiles(files));
    assert.deepEqual(Object.keys(back).sort(), ["First.svg", "Second.svg"]);
    assert.ok(strFromU8(back["First.svg"]).includes("<svg/>"));
  });

  console.log(` ${passed} passed, ${failed} failed`);
  if (failed > 0) process.exit(1);
}

void main();

=====
// === END FILE: tests/sidebar_batch.test.ts ===
=====

=====
// === START FILE: tests/sidebar_trash.test.ts ===
// File 182 of 194 | Lines: 164 | Size: 5723 bytes
=====

/**
 * Sidebar pro work: trash with undo.
 *
 * Deleting boards/folders from the sidebar must be undoable through the
 * store – restoring boards, folders, their pages AND their element data –
 * and the asset pruner must treat the undo buffer as referenced until the
 * undo window ends. Otherwise "Undo" brings back boards whose pictures and
 * PDFs were already reclaimed from IndexedDB.

```

```

*/

import assert from "node:assert/strict";
import { useWhiteboard } from "../src/store/whiteboard";
import { collectReferencedAssets } from "../src/lib/whiteboard/workspace";
import type { Board } from "../src/lib/whiteboard/types";

const NL = String.fromCharCode(10);

let passed = 0;
let failed = 0;
async function test(name: string, fn: () => void | Promise<void>) {
  try {
    await fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`      ${err as Error}.message`);
  }
}

function resetStore() {
  useWhiteboard.setState({
    boards: [],
    folders: [],
    activeBoardId: null,
    activeFolderId: null,
    selectedIds: [],
    editingId: null,
    history: { past: [], future: [] },
  });
  useWhiteboard.getState().discardTrash();
}

let seq = 0;
function board(partial: Partial<Board> = {}): Board {
  seq++;
  return {
    id: `b${seq}`,
    name: `Board ${seq}`,
    folderId: null,
    elements: [],
    background: { pattern: "plain" },
    camera: { x: 0, y: 0, zoom: 1 },
    createdAt: seq,
    updatedAt: seq,
    ...partial,
  } as Board;
}

function seed() {
  resetStore();
  const f1 = { id: "f1", name: "F1", parentId: null, color: "#fff", expanded: true, createdAt: 1 };
  const f2 = { id: "f2", name: "F2", parentId: "f1", color: "#fff", expanded: true, createdAt: 2 };
  const b1 = board({ id: "b1", name: "B1", folderId: null });
  const b2 = board({ id: "b2", name: "B2", folderId: "f1" });
  const b3 = board({
    id: "b3",
    name: "B3",
    folderId: "f2",
    kind: "notebook",
    pages: [
      {
        id: "p1",
        elements: [{ id: "e1", type: "text", x: 0, y: 0, rotation: 0, opacity: 1, text: "hi", createdAt: 1, up
          datedAt: 1 } as never],
        createdAt: 1,
        updatedAt: 1,
      },
    ],
  });
  useWhiteboard.setState({ folders: [f1, f2] as never, boards: [b1, b2, b3], activeBoardId: "b1" });
}

async function main() {
  console.log(NL + "███ Sidebar trash + undo ███");
}

```

```

await test("trashItems removes boards; undoTrash restores them in place", () => {
  seed();
  const S = () => useWhiteboard.getState();
  S().trashItems(["b1", "b3"], []);
  assert.deepEqual(S().boards.map((b) => b.id), ["b2"]);
  assert.ok(S().lastDeleted, "undo buffer holds the deletion");
  S().undoTrash();
  assert.deepEqual(S().boards.map((b) => b.id), ["b1", "b2", "b3"]);
  const b3 = S().boards.find((b) => b.id === "b3")!;
  assert.equal(b3.kind, "notebook");
  assert.equal((b3.pages ?? [])[0]?.elements.length, 1, "pages and elements survive");
  assert.equal(S().lastDeleted, null, "buffer clears after undo");
});

await test("trashing a folder removes its subtree but keeps boards at root", () => {
  seed();
  const S = () => useWhiteboard.getState();
  S().trashItems([], ["f1"]);
  assert.deepEqual(S().folders.map((f: { id: string }) => f.id), [], "subtree gone");
  assert.deepEqual(
    S().boards.map((b) => `${b.id}:${b.folderId ?? "root"}`).sort(),
    ["b1:root", "b2:root", "b3:root"]
  );
  S().undoTrash();
  assert.deepEqual(S().folders.map((f: { id: string }) => f.id).sort(), ["f1", "f2"]);
  const byId = new Map(S().boards.map((b) => [b.id, b.folderId]));
  assert.equal(byId.get("b2"), "f1", "board folder placement restored");
  assert.equal(byId.get("b3"), "f2");
});

await test("undoTrash with an empty buffer is a no-op", () => {
  seed();
  const S = () => useWhiteboard.getState();
  const before = S().boards.map((b) => b.id);
  S().undoTrash();
  assert.deepEqual(S().boards.map((b) => b.id), before);
});

await test("trashing unknown ids touches nothing and clobbers no buffer", () => {
  seed();
  const S = () => useWhiteboard.getState();
  S().trashItems(["nope"], ["missing"]);
  assert.deepEqual(S().boards.map((b) => b.id), ["b1", "b2", "b3"]);
  assert.equal(S().lastDeleted, null);
});

await test("a second trash replaces the first (single-level undo)", () => {
  seed();
  const S = () => useWhiteboard.getState();
  S().trashItems(["b1"], []);
  S().trashItems(["b2"], []);
  S().undoTrash();
  assert.deepEqual(S().boards.map((b) => b.id).sort(), ["b2", "b3"]);
});

await test("asset collection counts the undo buffer as referenced", () => {
  seed();
  const S = () => useWhiteboard.getState();
  const withAsset = board({
    id: "img1",
    elements: [{ id: "e9", type: "image", x: 0, y: 0, rotation: 0, opacity: 1, src: "asset:pic1", createdAt: 1, updatedAt: 1 } as never],
  });
  useWhiteboard.setState({ boards: [...S().boards, withAsset] });
  S().trashItems(["img1"], []);
  const buffered = (S().lastDeleted?.removedBoards ?? []).map((r) => r.board);
  const refs = collectReferencedAssets(S().boards, [], buffered);
  assert.ok(refs.imageIds.has("pic1"), "trashed board's picture stays referenced during undo window");
  S().discardTrash();
  const buffered2 = (S().lastDeleted?.removedBoards ?? []).map((r) => r.board);
  const refs2 = collectReferencedAssets(S().boards, [], buffered2);
  assert.ok(!refs2.imageIds.has("pic1"), "after discard the picture is reclaimable");
});

console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);
}

```

```
void main();

=====
// === END FILE: tests/sidebar_trash.test.ts ===
=====

=====
// === START FILE: tests/spen_deep_audit.test.ts ===
// File 183 of 194 | Lines: 514 | Size: 19930 bytes
=====

/**
 * S Pen Deep Debug & Audit Test Suite
 * =====
 * Exhaustive unit and integration test coverage for S Pen hardware responsiveness,
 * high-frequency 240Hz+ coordinate coalescence, drawing pointer ID isolation,
 * stylus barrel button transitions, auto-straighten spatial deadband, and all 8 pen engine geometries.
 */

import assert from "node:assert/strict";
import {
  calibratePressure,
  getStrokeOptionsForStyle,
  applyPenDynamics,
  isStrokeStraight,
  straightenStrokePoints,
  pruneTerminalHooks,
  getSvgPathFromStroke,
} from "../src/lib/whiteboard/stroke";
import {
  PEN_TYPE_PRESETS,
  type StrokeStyle,
} from "../src/lib/whiteboard/types";
import {
  stylusEraseTransition,
  isStylusEraseRequest,
  STYLUS_BARREL_BUTTON,
  STYLUS_ERASER_TIP,
  type StylusButtonState,
} from "../src/lib/whiteboard/input";
import {
  classifyTouch,
  contactLooksLikePalm,
  isRejectedTouch,
  type ContactGeometry,
  type PenProximity,
} from "../src/lib/whiteboard/palm";

let passed = 0;
let failed = 0;

function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(` ${err as Error}.message`);
  }
}

console.log("\n=====");
console.log(" ■■ S Pen Deep Debug & Inking Engine Verification Suite");
console.log("=====");

// =====
// 1. S Pen 240Hz+ Coalescence and getCoalescedEvents() Stream Processing
// =====
console.log("■■ 1. S Pen 240Hz+ Coalescence & Stream Processing ■■");

test("coalescence: processes sub-frame coalesced points maintaining chronological order", () => {
  // Simulate a 240Hz digitizer event with 4 coalesced samples in a 16.6ms display frame
  const coalescedEvents = [
    { clientX: 100, clientY: 100, pressure: 0.35, timeStamp: 1000 },
  ]
```

```

    { clientX: 104, clientY: 102, pressure: 0.38, timeStamp: 1004 },
    { clientX: 109, clientY: 105, pressure: 0.42, timeStamp: 1008 },
    { clientX: 115, clientY: 110, pressure: 0.46, timeStamp: 1012 },
  ];

  // Verify timestamps are strictly non-decreasing
  for (let i = 1; i < coalescedEvents.length; i++) {
    assert.ok(
      coalescedEvents[i].timeStamp > coalescedEvents[i - 1].timeStamp,
      "Coalesced points must preserve chronological order"
    );
  }

  // Convert to stroke point tuples
  const strokePoints: Array<[number, number, number]> = coalescedEvents.map((ev) => [
    ev.clientX,
    ev.clientY,
    calibratePressure(ev.pressure, 65, "standard"),
  ]);

  assert.equal(strokePoints.length, 4);
  assert.equal(strokePoints[0][0], 100);
  assert.equal(strokePoints[3][0], 115);
  // Pressure should be properly calibrated and monotonically increasing
  assert.ok(strokePoints[3][2] > strokePoints[0][2]);
});

test("coalescence: fallback to single primary event when getCoalescedEvents is unavailable", () => {
  const primaryEvent = { clientX: 150, clientY: 200, pressure: 0.5 };
  const rawCoalesced: Array<{ clientX: number; clientY: number; pressure: number }> = [];

  const eventsToProcess = rawCoalesced.length > 0 ? rawCoalesced : [primaryEvent];
  assert.equal(eventsToProcess.length, 1);
  assert.equal(eventsToProcess[0].clientX, 150);
});

test("coalescence: high-frequency 240Hz sampling prevents cursive loop faceting", () => {
  // Generate a tight cursive loop (e.g. cursive 'e') at 60Hz (4 points) vs 240Hz (16 points)
  const radius = 15;
  const cx = 50;
  const cy = 50;

  const points60Hz: Array<[number, number, number]> = [];
  for (let i = 0; i < 4; i++) {
    const angle = (i / 4) * Math.PI * 2;
    points60Hz.push([cx + Math.cos(angle) * radius, cy + Math.sin(angle) * radius, 0.5]);
  }

  const points240Hz: Array<[number, number, number]> = [];
  for (let i = 0; i < 16; i++) {
    const angle = (i / 16) * Math.PI * 2;
    points240Hz.push([cx + Math.cos(angle) * radius, cy + Math.sin(angle) * radius, 0.5]);
  }

  const path60 = getSvgPathFromStroke(points60Hz);
  const path240 = getSvgPathFromStroke(points240Hz);

  assert.ok(path60.startsWith("M") && path60.endsWith("Z"));
  assert.ok(path240.startsWith("M") && path240.endsWith("Z"));
  // 240Hz path contains more sub-pixel spline segments, rendering smooth loops
  assert.ok(path240.length > path60.length, "240Hz path should have higher geometric fidelity");
});

// =====
// 2. Drawing Pointer ID Isolation vs. Palm Interference
// =====
console.log("\n■■ 2. Drawing Pointer ID Isolation vs. Palm Interference ■■");

test("pointer isolation: inking is bound to active drawingPointerId", () => {
  let drawingPointerId: number | null = null;
  const strokePoints: Array<[number, number, number]> = [];

  // Stylus pointerdown
  const penDown = { pointerId: 42, pointerType: "pen", x: 100, y: 100, pressure: 0.5 };
  if (penDown.pointerType === "pen") {
    drawingPointerId = penDown.pointerId;
    strokePoints.push([penDown.x, penDown.y, penDown.pressure]);
  }

```

```
assert.equal(drawingPointerId, 42);

// Subsequent events from pen vs palm
const validPenMove = { pointerId: 42, pointerType: "pen", x: 110, y: 105, pressure: 0.5 };
const strayPalmMove = { pointerId: 99, pointerType: "touch", x: 400, y: 300, pressure: 0.1 };

// Event handler simulation with pointerId isolation
const handleMove = (ev: { pointerId: number; x: number; y: number; pressure: number }) => {
  if (ev.pointerId !== drawingPointerId) return; // Isolated!
  strokePoints.push([ev.x, ev.y, ev.pressure]);
};

handleMove(validPenMove);
handleMove(strayPalmMove); // Must be ignored!

assert.equal(strokePoints.length, 2);
assert.equal(strokePoints[1][0], 110);
assert.ok(!strokePoints.some((pt) => pt[0] === 400), "Palm coordinates must never be added to stroke");
});

test("pointer isolation: palm liftoff does not terminate active pen stroke", () => {
  let drawingPointerId: number | null = 42;
  let strokeCommitted = false;

  const handlePointerUp = (ev: { pointerId: number }) => {
    if (ev.pointerId !== drawingPointerId) return; // Palm lift ignored!
    drawingPointerId = null;
    strokeCommitted = true;
  };

  // Palm lifts off (pointerId 99) while pen (42) is still inking
  handlePointerUp({ pointerId: 99 });
  assert.equal(drawingPointerId, 42, "Drawing pointer ID must remain active");
  assert.equal(strokeCommitted, false, "Stroke must not be committed prematurely by palm lift");

  // Pen lifts off
  handlePointerUp({ pointerId: 42 });
  assert.equal(drawingPointerId, null);
  assert.equal(strokeCommitted, true, "Stroke committed when pen lifts");
});

test("pointer isolation: OS palm cancellation does not destroy active pen stroke", () => {
  let drawingPointerId: number | null = 42;
  let strokeCancelled = false;

  const handlePointerCancel = (ev: { pointerId: number }) => {
    if (ev.pointerId !== drawingPointerId) return; // Palm cancel ignored!
    drawingPointerId = null;
    strokeCancelled = true;
  };

  // Android OS sends ACTION_CANCEL for capacitive palm touch
  handlePointerCancel({ pointerId: 99 });
  assert.equal(drawingPointerId, 42, "Pen pointerId must remain intact when palm is cancelled");
  assert.equal(strokeCancelled, false);
});

test("palm classification: contact patch >= 45px classified as palm", () => {
  const palmGeometry: ContactGeometry = { width: 52, height: 38 };
  assert.equal(contactLooksLikePalm(palmGeometry), true);

  const fingerGeometry: ContactGeometry = { width: 18, height: 20 };
  assert.equal(contactLooksLikePalm(fingerGeometry), false);
});

test("palm classification: touch within PEN_SUPPRESSION_MS (800ms) after pen lift is rejected", () => {
  const now = 10000;
  const proxActive: PenProximity = { penDown: false, lastPenAt: now - 350, now }; // 350ms after lift
  const proxExpired: PenProximity = { penDown: false, lastPenAt: now - 950, now }; // 950ms after lift

  const verdictActive = classifyTouch({ width: 20, height: 20 }, proxActive);
  assert.equal(verdictActive, "pen-recent");
  assert.equal(isRejectedTouch({ width: 20, height: 20 }, proxActive), true);

  const verdictExpired = classifyTouch({ width: 20, height: 20 }, proxExpired);
  assert.equal(verdictExpired, "allow");
  assert.equal(isRejectedTouch({ width: 20, height: 20 }, proxExpired), false);
});
```

```
// =====
// 3. S Pen Barrel Button Transitions & State Machine
// =====
console.log("\n■■■ 3. S Pen Barrel Button Transitions & State Machine ■■■");

const pen = (buttons: number): StylusButtonState => ({ pointerType: "pen", buttons });
const NIB_CONTACT = 1;

test("barrel button: touchdown with button pressed engages eraser immediately", () => {
  const initialTouch = pen(NIB_CONTACT | STYLUS_BARREL_BUTTON); // nib down + button down
  assert.equal(isStylusEraseRequest(initialTouch), true);
  assert.equal(stylusEraseTransition(false, initialTouch), "engage");
});

test("barrel button: hover with button pressed arms eraser before contact", () => {
  const hoverTouch = pen(STYLUS_BARREL_BUTTON); // hovering, nib not touching
  assert.equal(stylusEraseTransition(false, hoverTouch), "engage");
});

test("barrel button: mid-stroke release restores inking without requiring pen liftoff", () => {
  // 1. Inking active
  let borrowed = false;
  assert.equal(stylusEraseTransition(borrowed, pen(NIB_CONTACT)), "none");

  // 2. User presses side button while sweeping
  const pressAction = stylusEraseTransition(borrowed, pen(NIB_CONTACT | STYLUS_BARREL_BUTTON));
  assert.equal(pressAction, "engage");
  borrowed = true;

  // 3. Sweeping eraser
  assert.equal(stylusEraseTransition(borrowed, pen(NIB_CONTACT | STYLUS_BARREL_BUTTON)), "none");

  // 4. User releases side button MID-STROKE (nib still down: buttons === 1)
  const releaseAction = stylusEraseTransition(borrowed, pen(NIB_CONTACT));
  assert.equal(releaseAction, "release");
  borrowed = false;

  // 5. Inking resumes immediately
  assert.equal(borrowed, false, "Pen tool is restored while nib is still touching glass");
});

test("barrel button: 80ms bounce grace window prevents eraser tool flickering", () => {
  // Simulating an 80ms grace window model for nib bounce during erasing
  class StylusEraseManager {
    isErasing = false;
    graceTimer: number | null = null;
    readonly GRACE_MS = 80;

    onPointerDown(hasButton: boolean) {
      if (this.graceTimer !== null) {
        // Cancel pending release if re-contact occurs within grace window
        this.graceTimer = null;
      }
      if (hasButton) {
        this.isErasing = true;
      }
    }

    onPointerUp(currentTime: number) {
      if (this.isErasing) {
        // Arm grace timer
        this.graceTimer = currentTime + this.GRACE_MS;
      }
    }

    checkGraceExpiry(currentTime: number) {
      if (this.graceTimer !== null && currentTime >= this.graceTimer) {
        this.isErasing = false;
        this.graceTimer = null;
      }
    }
  }

  const manager = new StylusEraseManager();
  manager.onPointerDown(true);
  assert.equal(manager.isErasing, true);
});
```

```

// Micro nib bounce: pen lifts at t = 100ms
manager.onPointerUp(100);
assert.equal(manager.isErasing, true, "Eraser remains engaged during grace window");

// Nib touches back down at t = 130ms (30ms later < 80ms grace)
manager.onPointerDown(true);
assert.equal(manager.isErasing, true, "Eraser did not flicker to pen");
assert.equal(manager.graceTimer, null, "Grace timer cancelled on re-contact");

// Pen lifts at t = 200ms and stays off past grace window
manager.onPointerUp(200);
manager.checkGraceExpiry(290); // 90ms later > 80ms grace
assert.equal(manager.isErasing, false, "Eraser released after grace window expires");
});

test("barrel button: exhaustive transition matrix consistency across all 64 bitmasks", () => {
  for (let b = 0; b < 64; b++) {
    const isErase = (b & STYLUS_BARREL_BUTTON) !== 0 || (b & STYLUS_ERASER_TIP) !== 0;
    const req = isStylusEraseRequest(pen(b));
    assert.equal(req, isErase);

    const fromIdle = stylusEraseTransition(false, pen(b));
    assert.equal(fromIdle, isErase ? "engage" : "none");

    const fromBorrowed = stylusEraseTransition(true, pen(b));
    assert.equal(fromBorrowed, isErase ? "none" : "release");
  }
});

// =====
// 4. Draw-and-Hold Auto-Straightening Spatial Deadband
// =====
console.log("\n■■■ 4. Draw-and-Hold Auto-Straightening Spatial Deadband ■■■");

test("auto-straighten: detects intentional straight line with chord ratio >= 0.88", () => {
  const linePoints: Array<[number, number, number]> = [
    [0, 100, 0.5],
    [25, 101, 0.5],
    [50, 99, 0.5],
    [75, 100, 0.5],
    [100, 100, 0.5],
    [125, 101, 0.5],
  ];
  assert.ok(isStrokeStraight(linePoints, 0.88), "Near-straight line must be detected");
});

test("auto-straighten: rejects short strokes with chord < 28px", () => {
  const shortPoints: Array<[number, number, number]> = [
    [0, 0, 0.5],
    [5, 0, 0.5],
    [10, 0, 0.5],
    [15, 0, 0.5],
    [20, 0, 0.5],
  ];
  assert.ok(!isStrokeStraight(shortPoints, 0.88), "Stroke < 28px must not auto-straighten");
});

test("auto-straighten: spatial deadband (5px) qualifies micro-jitter vs intentional movement", () => {
  const HOLD_DEADBAND_PX = 5;

  const anchor = { x: 100, y: 100 };
  const microJitter = { x: 102.5, y: 101.8 }; // Dist ~ 3.08px (< 5px)
  const intentionalMove = { x: 108.0, y: 100.0 }; // Dist = 8px (> 5px)

  const distJitter = Math.hypot(microJitter.x - anchor.x, microJitter.y - anchor.y);
  const distMove = Math.hypot(intentionalMove.x - anchor.x, intentionalMove.y - anchor.y);

  assert.ok(distJitter <= HOLD_DEADBAND_PX, "Micro-jitter must fall within deadband");
  assert.ok(distMove > HOLD_DEADBAND_PX, "Intentional move must exceed deadband");

  // In the event loop, deadband check avoids timer reset:
  const shouldResetHoldTimer = (currentX: number, currentY: number) => {
    return Math.hypot(currentX - anchor.x, currentY - anchor.y) > HOLD_DEADBAND_PX;
  };

  assert.equal(shouldResetHoldTimer(microJitter.x, microJitter.y), false);
  assert.equal(shouldResetHoldTimer(intentionalMove.x, intentionalMove.y), true);
});

```

```

test("auto-straighten: rubber-banding interpolates 8 points preserving average pressure", () => {
  const strokePoints: Array<[number, number, number]> = [
    [0, 0, 0.2],
    [50, 0, 0.4],
    [100, 0, 0.6],
    [150, 0, 0.8],
  ];
  const avgPressure = (0.2 + 0.4 + 0.6 + 0.8) / 4; // 0.5

  const straightened = straightenStrokePoints(strokePoints, 8);
  assert.equal(straightened.length, 9);
  assert.equal(straightened[0][0], 0);
  assert.equal(straightened[8][0], 150);

  for (const pt of straightened) {
    assert.equal(pt[2], avgPressure, "All interpolated points must have uniform average pressure");
  }
});

// =====
// 5. All 8 Pen Engine Geometries & Nib Physics
// =====
console.log("\n■ 5. All 8 Pen Engine Geometries & Nib Physics ■");

test("pen engines: all 8 presets configured with distinct physical characteristics", () => {
  const requiredEngines: StrokeStyle[] = [
    "ballpoint",
    "gel",
    "fineliner",
    "fountain",
    "brush",
    "marker",
    "highlighter",
    "pencil",
  ];

  for (const eng of requiredEngines) {
    const p = PEN_TYPE_PRESETS[eng];
    assert.ok(p, `Preset for ${eng} must exist`);
    assert.ok(p.label, `Label for ${eng} must exist`);
  }

  // Chisel caps
  assert.equal(PEN_TYPE_PRESETS.marker.cap, "flat");
  assert.equal(PEN_TYPE_PRESETS.highlighter.cap, "flat");
  assert.equal(PEN_TYPE_PRESETS.ballpoint.cap, "round");

  // Blend modes
  assert.equal(PEN_TYPE_PRESETS.highlighter.mixBlendMode, "multiply");
  assert.equal(PEN_TYPE_PRESETS.pencil.mixBlendMode, "multiply");
  assert.equal(PEN_TYPE_PRESETS.ballpoint.mixBlendMode, "normal");

  // Pencil grain
  assert.equal(PEN_TYPE_PRESETS.pencil.grain, true);

  // Brush pressure simulation
  assert.equal(PEN_TYPE_PRESETS.brush.simulatePressure, true);
  assert.ok(PEN_TYPE_PRESETS.brush.startTaper > 0 && PEN_TYPE_PRESETS.brush.endTaper > 0);
});

test("pressure calibration: single calibration preserves curve response across range", () => {
  // Test sensitivity = 50 (unscaled curve)
  assert.ok(Math.abs(calibratePressure(0.25, 50, "soft") - Math.pow(0.25, 0.7)) < 1e-6);
  assert.ok(Math.abs(calibratePressure(0.25, 50, "firm") - Math.pow(0.25, 1.4)) < 1e-6);
  assert.equal(calibratePressure(0.25, 50, "off"), 0.5);
});

test("fountain nib: chisel angle (40 deg) produces directional thickness variation", () => {
  // Along the nib (travel angle = 40 deg): across = |sin(40 - 40)| = 0 -> min pressure
  // Across the nib (travel angle = 130 deg): across = |sin(130 - 40)| = 1 -> max pressure
  const rad40 = (40 * Math.PI) / 180;
  const rad130 = (130 * Math.PI) / 180;

  const strokeAlong: Array<[number, number, number]> = [
    [0, 0, 0.5],
    [Math.cos(rad40) * 10, Math.sin(rad40) * 10, 0.5],
  ];
};

```

```

const strokeAcross: Array<[number, number, number]> = [
  [0, 0, 0.5],
  [Math.cos(rad130) * 10, Math.sin(rad130) * 10, 0.5],
];

const dynAlong = applyPenDynamics(strokeAlong, "fountain");
const dynAcross = applyPenDynamics(strokeAcross, "fountain");

assert.ok(
  dynAlong[1][2] < dynAcross[1][2],
  `Stroke along nib (${dynAlong[1][2]}) must be thinner than across nib (${dynAcross[1][2]})`
);
});

test("hook pruning: sharp lift-off flick is pruned while deliberate checkmark flourish is preserved", () => {
  // Flick that reverses direction by > 95 degrees at stroke end
  const hookStroke: Array<[number, number, number]> = [
    [0, 0, 0.5],
    [20, 0, 0.5],
    [40, 0, 0.5],
    [60, 0, 0.5],
    [80, 0, 0.5],
    [65, 5, 0.15], // sharp reverse hook flick
  ];
  const pruned = pruneTerminalHooks(hookStroke);
  assert.equal(pruned.length, 5, "Terminal hook flick should be pruned");

  // Deliberate checkmark flourish (smooth turn, not a reverse hook flick)
  const checkmarkStroke: Array<[number, number, number]> = [
    [0, 50, 0.5],
    [20, 70, 0.5],
    [40, 90, 0.6],
    [60, 60, 0.5],
    [80, 30, 0.4],
  ];
  const preserved = pruneTerminalHooks(checkmarkStroke);
  assert.equal(preserved.length, 5, "Intentional checkmark must be preserved");
});

test("dynamic taper scaling: short stroke tapers do not exceed 25% of total arc length", () => {
  // A very short stroke (length = 10px) with brush preset (startTaper: 6, endTaper: 14)
  const shortBrushOptions = getStrokeOptionsForStyle("brush", 8);
  const totalArcLength = 10;
  const maxTaper = totalArcLength * 0.25; // 2.5px clamp

  const clampedStartTaper = Math.min(shortBrushOptions.start?.taper ?? 0, maxTaper);
  const clampedEndTaper = Math.min(shortBrushOptions.end?.taper ?? 0, maxTaper);

  assert.ok(clampedStartTaper <= 2.5, "Start taper on short stroke must not exceed 25% arc length");
  assert.ok(clampedEndTaper <= 2.5, "End taper on short stroke must not exceed 25% arc length");
});

console.log("\n=====");
console.log(` S Pen Deep Debug Audit: ${passed} passed, ${failed} failed`);
console.log("=====");

if (failed > 0) {
  process.exit(1);
}

=====
// === END FILE: tests/spen_deep_audit.test.ts ===
=====

=====
// === START FILE: tests/spen_review.test.ts ===
// File 184 of 194 | Lines: 114 | Size: 4512 bytes
=====

/**
 * Regressions from the S Pen review:
 * - Pressure is calibrated once, when captured - never again when drawn, so
 *   strokes already in a notebook look exactly as they did when written.
 * - The default pressure setting is neutral: new strokes match older ones.
 */

import assert from "node:assert/strict";

```

```

import { applyPenDynamics, buildStrokePath, calibratePressure } from "../src/lib/whiteboard/stroke";
import { DEFAULT_TOUCH_SETTINGS, PEN_TYPES, penFeelFor, penUsesPressure, type StrokeElement, type StrokeStyle
  } from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`   ${err as Error}.message`);
  }
}

console.log(String.fromCharCode(10) + "■ S Pen review ■");

/** A stroke as an older build saved it: raw pressure, no smoothing field. */
function oldStroke(style: StrokeStyle): StrokeElement {
  const points: Array<number, number, number> = [];
  for (let i = 0; i < 40; i++) points.push([i * 3, Math.sin(i / 4) * 12, 0.15 + (i % 10) * 0.08]);
  return {
    id: "old",
    type: "stroke",
    x: 0,
    y: 0,
    rotation: 0,
    opacity: 1,
    createdAt: 0,
    updatedAt: 0,
    points,
    color: "#000",
    strokeWidth: 6,
    strokeStyle: style,
  } as StrokeElement;
}

test("drawing a stroke leaves its stored pressure alone (no second calibration)", () => {
  const pts: Array<number, number, number> = [
    [0, 0, 0.2],
    [0.5, 0, 0.8],
    [1, 0, 0.35],
  ];
  // Fineliner has no speed or nib dynamics: every pressure must pass through.
  assert.deepEqual(applyPenDynamics(pts, "fineliner"), pts);
  // Gel only thins with speed; at a crawl the pressure is essentially kept.
  const gel = applyPenDynamics(pts, "gel");
  gel.forEach((p, i) => assert.ok(Math.abs(p[2] - pts[i][2]) < 0.01, `gel point ${i}: ${p[2]} vs ${pts[i][2]}`));
});

test("strokes written before the pressure settings existed render unchanged", () => {
  // The same stroke drawn with the default smoothing must give the same path
  // whether or not the element carries the new field.
  for (const st of PEN_TYPES) {
    const a = buildStrokePath(oldStroke(st));
    const b = buildStrokePath({ ...oldStroke(st), smoothing: 50 });
    assert.equal(a, b, `${st} changed`);
  }
});

test("the default pressure setting is neutral: the curve exactly, no widening", () => {
  assert.equal(DEFAULT_TOUCH_SETTINGS.penPressureSensitivity, 50);
  assert.equal(DEFAULT_TOUCH_SETTINGS.penPressureCurve, "standard");
  for (const p of [0.05, 0.2, 0.5, 0.73, 0.95]) {
    const out = calibratePressure(p, DEFAULT_TOUCH_SETTINGS.penPressureSensitivity, DEFAULT_TOUCH_SETTINGS.penPressureCurve);
    assert.ok(Math.abs(out - p) < 1e-9, `default setting moved ${p} to ${out}`);
  }
});

test("pressure curves still shape the response in the right direction", () => {
  const soft = calibratePressure(0.3, 50, "soft");
  const firm = calibratePressure(0.3, 50, "firm");
  assert.ok(soft > 0.3 && firm < 0.3, `soft ${soft}, firm ${firm}`);
});

```

```

    assert.equal(calibratePressure(0.9, 50, "off"), 0.5);
  });

test("the picker offers six pens that look different; old pens still draw", () => {
  assert.deepEqual(PEN_TYPES, ["gel", "fountain", "brush", "pencil", "marker", "highlighter"]);
  for (const legacy of ["ballpoint", "fineliner"] as StrokeStyle[]) {
    assert.ok(buildStrokePath(oldStroke(legacy)).length > 0, `${legacy} strokes must still render`);
  }
});

test("pressure settings are offered only to pens that react to pressure", () => {
  assert.equal(penUsesPressure("gel"), true);
  assert.equal(penUsesPressure("fountain"), true);
  assert.equal(penUsesPressure("pencil"), true);
  assert.equal(penUsesPressure("brush"), false, "brush width comes from speed");
  assert.equal(penUsesPressure("marker"), false);
  assert.equal(penUsesPressure("highlighter"), false);
});

test("each pen keeps its own feel; untuned pens use the shared one", () => {
  const ts = {
    ...DEFAULT_TOUCH_SETTINGS,
    penSmoothing: 40,
    penFeel: { fountain: { curve: "firm" as const, smoothing: 80 } },
  };
  assert.deepEqual(penFeelFor(ts, "fountain"), { curve: "firm", smoothing: 80, sensitivity: 50 });
  assert.deepEqual(penFeelFor(ts, "gel"), { curve: "standard", smoothing: 40, sensitivity: 50 });
});

console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/spen_review.test.ts ===
=====

=====
// === START FILE: tests/split_view.test.ts ===
// File 185 of 194 | Lines: 435 | Size: 16237 bytes
=====

/**
 * Split view: switching which side is live must change nothing you can see,
 * and must not cost you your undo.
 *
 * Both were broken, and both were reproduced on a real build before being
 * fixed:
 *
 * - Tapping the other pane moved it. Its camera went from y 561 to 640,
 *   because the live canvas clamps a notebook's camera to the paper and the
 *   other pane did not - so every switch was followed by a "correction".
 * - Reading page 5 in the other pane and tapping it put you on page 8. That
 *   pane kept a private page counter while the notebook's own active page
 *   never moved.
 * - Every switch wiped the undo history, because a switch is a board change
 *   and setActiveBoard emptied the whole list.
 *
 * The guarantee that makes "nothing moves" hold is idempotence: the other pane
 * shows the stored camera PUT THROUGH the live canvas's rules, so when the
 * canvas takes over and applies those same rules, there is nothing left to
 * change. Those properties are pinned directly below.
 */

import assert from "node:assert/strict";

import { useWhiteboard } from "../src/store/whiteboard";
import {
  clampNotebookCamera,
  getActivePageIndex,
  getPages,
  minNotebookZoom,
  PAGE_GAP,
  reanchorDirection,
  visibleSheets,
  allElements,
} from "../src/lib/whiteboard/notebook";
import { cleanHistoryEntry, migrateState } from "../src/lib/whiteboard/persistence";

```

```

import { dividerRatio, MIN_PANE_PX } from "../src/lib/whiteboard/split";
import type { Board, Camera, NotebookPage, WhiteboardElement } from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`${(err as Error).message}`);
  }
}

const S = () => useWhiteboard.getState();
const board = (id: string): Board => S().boards.find((b) => b.id === id)!;

const A4 = { width: 794, height: 1123 };
const PANE = { width: 512, height: 800 };

let seq = 0;
function text(label: string): WhiteboardElement {
  seq++;
  return {
    id: `t${seq}`,
    type: "text",
    x: 60,
    y: 120,
    width: 300,
    height: 40,
    rotation: 0,
    opacity: 1,
    text: label,
    color: "#111",
    fontSize: 24,
    fontFamily: "Inter",
    fontWeight: 400,
    align: "left",
    createdAt: 1,
    updatedAt: 1,
  } as unknown as WhiteboardElement;
}

function notebook(id: string, pageCount: number, camera: Camera = { x: 397, y: 561, zoom: 0.5 }): Board {
  const pages: NotebookPage[] = Array.from({ length: pageCount }, (_, i) => ({
    id: `${id}_p${i + 1}`,
    elements: [text(`${id} page ${i + 1}`)],
    createdAt: 1,
    updatedAt: 1,
  }));
  return {
    id,
    name: id,
    folderId: null,
    elements: [],
    camera,
    kind: "notebook",
    pages,
    activePageId: pages[0].id,
    pageSize: "a4",
    createdAt: 1,
    updatedAt: 1,
  } as Board;
}

/** Two notebooks in split view: `left` live, `right` in the other pane. */
function split(left: Board, right: Board) {
  useWhiteboard.setState({
    boards: [left, right],
    activeBoardId: left.id,
    camera: { ...left.camera },
    splitBoardId: right.id,
    splitSide: "left",
    splitRatio: 0.5,
    selectedIds: [],
  });
}

```

```

    editingId: null,
    history: { past: [], future: [] },
  });
}

/** The live canvas's settle rules, in the order it applies them. */
function settle(cam: Camera, b: Board, vp = PANE): Camera {
  const floor = minNotebookZoom(vp, A4);
  const zoom = floor > 0 && cam.zoom < floor - 1e-3 ? floor : cam.zoom;
  const { x, y } = clampNotebookCamera({ ...cam, zoom }, vp, A4, {
    pageCount: getPages(b).length,
    activeIndex: getActivePageIndex(b),
  });
  return { x, y, zoom };
}

console.log("\n■■ One layout for every pane ■■");

test("visibleSheets reproduces the live canvas's own neighbour rule", () => {
  // The canvas's inline logic, restated: every page a stride apart relative to
  // the active one, skipping anything more than a stride outside the band.
  const reference = (count: number, active: number, camY: number, halfH: number) => {
    const stride = A4.height + PAGE_GAP;
    const out: Array<{ index: number; offsetY: number }> = [];
    for (let i = 0; i < count; i++) {
      const offsetY = (i - active) * stride;
      if (offsetY + A4.height < camY - halfH - stride) continue;
      if (offsetY > camY + halfH + stride) continue;
      out.push({ index: i, offsetY });
    }
    return out;
  };
  for (const [count, active, camY, halfH] of [
    [8, 0, 561, 800],
    [8, 4, 200, 400],
    [8, 7, 1100, 1600],
    [200, 120, 561, 800],
    [1, 0, 0, 800],
  ]) {
    assert.deepEqual(visibleSheets(count, active, A4.height, camY, halfH), reference(count, active, camY, halfH));
  }
});

test("a 200-page notebook draws a handful of sheets, not 200", () => {
  const sheets = visibleSheets(200, 100, A4.height, 561, 800);
  assert.ok(sheets.length > 0 && sheets.length <= 5, `drew ${sheets.length}`);
  assert.ok(sheets.some(s => s.index === 100 && s.offsetY === 0), "the anchored page sits at y = 0");
});

test("re-anchoring hands the page on at the middle of the gap, both ways", () => {
  const h = A4.height;
  assert.equal(reanchorDirection(h + PAGE_GAP / 2 + 1, h, 2, 8), 1);
  assert.equal(reanchorDirection(h + PAGE_GAP / 2 - 1, h, 2, 8), 0);
  assert.equal(reanchorDirection(-PAGE_GAP / 2 - 1, h, 2, 8), -1);
  assert.equal(reanchorDirection(-PAGE_GAP / 2 + 1, h, 2, 8), 0);
  assert.equal(reanchorDirection(99999, h, 7, 8), 0, "no page after the last one");
  assert.equal(reanchorDirection(-99999, h, 0, 8), 0, "no page before the first one");
});

console.log("\n■■ Switching must move nothing ■■");

test("settling is idempotent - the property that makes a switch invisible", () => {
  // The other pane shows settle(stored). When the canvas takes over it applies
  // settle() again. If a second pass changed anything, the page would jump.
  const b = notebook("N", 8);
  for (const cam of [
    { x: 397, y: 561, zoom: 0.5 },
    { x: 0, y: -900, zoom: 0.2 },
    { x: 2000, y: 9000, zoom: 3 },
    { x: 397, y: 284.77, zoom: 0.7167 },
  ]) {
    const once = settle(cam, b);
    const twice = settle(once, b);
    assert.deepEqual(twice, once, `second pass moved ${JSON.stringify(cam)}`);
  }
});

```

```

test("the camera that used to jump on a switch is already settled when shown", () => {
  // Reproduced on the build: stored y 561 was shown at 561 in the other pane,
  // then corrected to 640 the moment the canvas took over.
  const b = notebook("N", 8);
  const shown = settle({ x: 397, y: 561, zoom: 0.5 }, b);
  assert.equal(Math.round(shown.y), 640, "shown where the canvas would put it");
  assert.deepEqual(settle(shown, b), shown, "so the canvas has nothing to correct");
});

test("focusing the other pane leaves both documents' cameras exactly as they were", () => {
  const L = notebook("L", 8, { x: 397, y: 284.77, zoom: 0.7167 });
  const R = notebook("R", 8, { x: 397, y: 640, zoom: 0.5 });
  split(L, R);
  const before = { L: { ...S().camera }, R: { ...board("R").camera } };

  S().focusSplitSide("right");
  assert.equal(S().activeBoardId, "R", "the other pane is now live");
  assert.deepEqual(S().camera, before.R, "R shows exactly what it showed before");
  assert.deepEqual(board("L").camera, before.L, "L keeps its view as it becomes the other pane");

  S().focusSplitSide("left");
  assert.deepEqual(S().camera, before.L, "and back again, still untouched");
  assert.deepEqual(board("R").camera, before.R);
});

console.log("\n■■ The page you read is the page you write on ■■");

test("turning the other pane's page moves THAT notebook's own page", () => {
  const L = notebook("L", 8);
  const R = notebook("R", 8);
  split(L, R);
  S().goToBoardPage("R", "R_p5", PANE.height);
  assert.equal(getActivePageIndex(board("R")), 4, "R is on page 5");
  assert.equal(getActivePageIndex(board("L")), 0, "the live notebook did not move");
  assert.equal(S().activeBoardId, "L", "turning a page does not by itself switch the store");
});

test("...so switching to it opens on page 5, not wherever it was before", () => {
  const L = notebook("L", 8);
  const R = notebook("R", 8);
  split(L, R);
  S().goToBoardPage("R", "R_p5", PANE.height);
  S().focusSplitSide("right");
  assert.equal(getActivePageIndex(board("R")), 4, "still page 5 once live");
});

test("the page opens at its top, framed like the live canvas's own page buttons", () => {
  const L = notebook("L", 8);
  const R = notebook("R", 8, { x: 397, y: 561, zoom: 0.5 });
  split(L, R);
  S().goToBoardPage("R", "R_p3", PANE.height);
  // goToPage's framing: the page top 24 px below the top of the pane.
  assert.equal(board("R").camera.y, PANE.height / 2 / 0.5 - 24 / 0.5);
});

test("scrolling the other pane across a page hands the anchor on without moving", () => {
  const L = notebook("L", 8);
  const R = notebook("R", 8, { x: 397, y: 1200, zoom: 0.5 });
  split(L, R);
  const stride = A4.height + PAGE_GAP;
  S().reanchorBoardPage("R", 1, stride);
  assert.equal(getActivePageIndex(board("R")), 1, "page 2 is now the anchor");
  assert.equal(board("R").camera.y, 1200 - stride, "and the camera slid one stride to match");
  assert.equal(getActivePageIndex(board("L")), 0, "the live notebook is untouched");
});

console.log("\n■■ Undo survives switching sides ■■");

test("switching panes no longer wipes the undo history", () => {
  const L = notebook("L", 2);
  const R = notebook("R", 2);
  split(L, R);
  S().addElement(text("drawn on L"), true);
  const depth = S().history.past.length;
  assert.ok(depth > 0);
  S().focusSplitSide("right");
  S().focusSplitSide("left");
  assert.equal(S().history.past.length, depth, "still there after going across and back");
});

```

```

});

test("undo on the live side undoes the live side's work", () => {
  const L = notebook("L", 2);
  const R = notebook("R", 2);
  split(L, R);
  S().addElement(text("on L"), true);
  const lAfter = allElements(board("L")).length;
  S().focusSplitSide("right");
  S().addElement(text("on R"), true);
  const rAfter = allElements(board("R")).length;

  S().undo();
  assert.equal(allElements(board("R")).length, rAfter - 1, "R's stroke came off");
  assert.equal(allElements(board("L")).length, lAfter, "L was not touched");
});

test("undo never reaches across the divider into the pane you are not in", () => {
  const L = notebook("L", 2);
  const R = notebook("R", 2);
  split(L, R);
  S().addElement(text("on L"), true);
  const lAfter = allElements(board("L")).length;
  S().focusSplitSide("right");
  // Nothing done on R yet: undo here must do nothing at all.
  S().undo();
  S().undo();
  assert.equal(allElements(board("L")).length, lAfter, "L's work is safe while you are on R");
  assert.equal(S().canUndo(), false, "and the UI can say there is nothing to undo here");
});

test("going back to the other side brings its undo back with it", () => {
  const L = notebook("L", 2);
  const R = notebook("R", 2);
  split(L, R);
  const lStart = allElements(board("L")).length;
  S().addElement(text("on L"), true);
  S().focusSplitSide("right");
  S().addElement(text("on R"), true);
  S().focusSplitSide("left");
  S().undo();
  assert.equal(allElements(board("L")).length, lStart, "L's own edit is undone on L");
});

test("a new edit on one side keeps the other side's redo", () => {
  const L = notebook("L", 2);
  const R = notebook("R", 2);
  split(L, R);
  S().focusSplitSide("right");
  S().addElement(text("on R"), true);
  S().undo();
  assert.equal(S().canRedo(), true, "R has something to redo");
  S().focusSplitSide("left");
  S().addElement(text("on L"), true); // a fresh edit on L...
  S().focusSplitSide("right");
  assert.equal(S().canRedo(), true, "...did not throw away R's redo");
});

test("merging into a closed notebook is still undone from where you did it", () => {
  // The Duplicate dialog's promise: one Ctrl+Z cancels the merge, even though
  // the pages changed are in a notebook that is not on screen.
  const here = notebook("HERE", 2);
  const closed = notebook("CLOSED", 2);
  const source = notebook("SRC", 3);
  useWhiteboard.setState({
    boards: [here, closed, source],
    activeBoardId: "HERE",
    camera: { ...here.camera },
    splitBoardId: null,
    selectedIds: [],
    editingId: null,
    history: { past: [], future: [] },
  });
  S().insertPagesFromBoard("SRC", { indices: [1] }, 2, "CLOSED");
  assert.equal(getPages(board("CLOSED")).length, 3);
  S().undo();
  assert.equal(getPages(board("CLOSED")).length, 2, "one undo from HERE cancelled it");
});

```

```

test("switching notebooks from the sidebar never lets undo edit one you cannot see", () => {
  const A = notebook("A", 1);
  const B = notebook("B", 1);
  useWhiteboard.setState({
    boards: [A, B],
    activeBoardId: "A",
    camera: { ...A.camera },
    splitBoardId: null,
    selectedIds: [],
    editingId: null,
    history: { past: [], future: [] },
  });
  S().addElement(text("on A"), true);
  const aAfter = allElements(board("A")).length;
  S().setActiveBoard("B");
  S().undo();
  assert.equal(allElements(board("A")).length, aAfter, "A, off screen, was not changed");
});

console.log("\n■■ Older saves still load ■■");

test("history entries saved before `activeAt` existed still load and still undo", () => {
  const legacy = { boardId: "A", elements: [], at: 5 };
  const clean = cleanHistoryEntry(legacy);
  assert.ok(clean, "accepted");
  assert.equal(clean!.activeAt, undefined, "no context recorded - it means the board itself");
});

test("the recorded context survives a save and reload", () => {
  const clean = cleanHistoryEntry({ boardId: "CLOSED", activeAt: "HERE", elements: [], at: 5 });
  assert.equal(clean!.activeAt, "HERE");
});

// -----
// Top & bottom
// -----

test("the divider keeps both panes big enough for the tools", () => {
  // 800px tall tablet, stacked: neither pane below MIN_PANE_PX.
  assert.equal(dividerRatio(10, 800), MIN_PANE_PX / 800);
  assert.equal(dividerRatio(790, 800), 1 - MIN_PANE_PX / 800);
  assert.equal(dividerRatio(400, 800), 0.5);
  // A wide row still allows 20/80.
  assert.equal(dividerRatio(0, 2000), 0.2);
  // A tiny row can't honour the minimum twice: it stays near the middle.
  assert.ok(Math.abs(dividerRatio(0, 300) - 0.45) < 1e-9);
  assert.equal(dividerRatio(5, 0), 0.5);
});

test("the layout can be picked when opening a split and switched later", () => {
  const s = useWhiteboard.getState();
  const a = s.createBoard("Stack A", null);
  const b = s.createBoard("Stack B", null);
  useWhiteboard.getState().setSplitPair(a, b, "stacked");
  let st = useWhiteboard.getState();
  assert.equal(st.splitOrientation, "stacked");
  assert.equal(st.activeBoardId, a, "the first (top) one is editable");
  assert.equal(st.splitBoardId, b);
  // Switching the layout keeps the same documents in the same slots.
  useWhiteboard.getState().setSplitOrientation("side");
  st = useWhiteboard.getState();
  assert.equal(st.splitOrientation, "side");
  assert.equal(st.activeBoardId, a);
  assert.equal(st.splitSide, "left");
  // Swapping keeps the layout.
  useWhiteboard.getState().setSplitOrientation("stacked");
  useWhiteboard.getState().swapSplit();
  assert.equal(useWhiteboard.getState().splitOrientation, "stacked");
  assert.equal(useWhiteboard.getState().splitSide, "right");
  useWhiteboard.getState().setSplitBoard(null);
});

test("the layout survives a save and reload; old saves open side by side", () => {
  const base = {
    version: 2,
    boards: [],
    activeBoardId: null,
  };

```

```

    activeFolderId: null,
    folders: [],
  } as unknown as Parameters<typeof migrateState>[0];
  assert.equal(migrateState({ ...base, splitOrientation: "stacked" }).splitOrientation, "stacked");
  assert.equal(migrateState(base).splitOrientation, "side");
  assert.equal(migrateState({ ...base, splitOrientation: "diagonal" as never }).splitOrientation, "side");
});

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/split_view.test.ts ===
=====

=====
// === START FILE: tests/stylus_button.test.ts ===
// File 186 of 194 | Lines: 116 | Size: 4788 bytes
=====

/**
 * Stylus barrel button -> hold to erase, release to get your pen back.
 *
 * The bug this pins: the first implementation read the button on pointerdown
 * and handed the pen back on pointerup, so the eraser was keyed to the NIB.
 * You had to touch the paper before the eraser appeared, and once it did you
 * were stuck in it until you lifted off - releasing the button changed
 * nothing. Holding the button while hovering did nothing at all.
 *
 * stylusEraseTransition reports EDGES on the button instead, which is what
 * lets the canvas run the same check from pointerdown and pointermove alike.
 */

import assert from "node:assert/strict";

import {
  stylusEraseTransition,
  isStylusEraseRequest,
  STYLUS_BARREL_BUTTON,
  STYLUS_ERASER_TIP,
  type StylusButtonState,
} from "../src/lib/whiteboard/input";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(` ${err as Error}.message`);
  }
}

const pen = (buttons: number): StylusButtonState => ({ pointerType: "pen", buttons });
const NIB = 1; // primary "button" = the tip is on the glass

console.log(`\n■■ Stylus button erase ■■`);

test("button held while merely hovering arms the eraser", () => {
  // buttons === 2 with no nib bit: pen in the air, side button down.
  assert.equal(stylusEraseTransition(false, pen(STYLUS_BARREL_BUTTON)), "engage");
});

test("button held with the nib down also arms it", () => {
  assert.equal(stylusEraseTransition(false, pen(NIB | STYLUS_BARREL_BUTTON)), "engage");
});

test("the eraser END of the pen arms it too", () => {
  assert.equal(stylusEraseTransition(false, pen(STYLUS_ERASER_TIP)), "engage");
});

test("releasing the button gives the pen back WITH THE NIB STILL DOWN", () => {
  // This is the whole point: mid-stroke, button up, nib still on the glass.

```

```

    assert.equal(stylusEraseTransition(true, pen(NIB)), "release");
  });

test("releasing the button while hovering gives the pen back", () => {
  assert.equal(stylusEraseTransition(true, pen(0)), "release");
});

test("holding the button does not re-engage every event", () => {
  // Re-engaging would overwrite the stored tool with "eraser" and strand the
  // user there forever - the exact trap the `borrowed` flag exists to avoid.
  assert.equal(stylusEraseTransition(true, pen(STYLUS_BARREL_BUTTON)), "none");
});

test("an idle pen with no button does nothing", () => {
  assert.equal(stylusEraseTransition(false, pen(0)), "none");
  assert.equal(stylusEraseTransition(false, pen(NIB)), "none");
});

test("a finger never releases an eraser the pen is holding", () => {
  // Palm and finger events interleave with pen events on a tablet. If touch
  // could answer for the pen, resting a hand mid-erase would snap the tool
  // back underneath you.
  assert.equal(stylusEraseTransition(true, { pointerType: "touch", buttons: 0 }), "none");
  assert.equal(stylusEraseTransition(true, { pointerType: "touch", buttons: 1 }), "none");
});

test("a mouse never engages it", () => {
  // buttons === 2 on a mouse is right-click, which must stay a context menu.
  assert.equal(stylusEraseTransition(false, { pointerType: "mouse", buttons: 2 }), "none");
  assert.equal(stylusEraseTransition(true, { pointerType: "mouse", buttons: 0 }), "none");
});

test("transition agrees with isStylusEraseRequest on every button mask", () => {
  for (let b = 0; b < 64; b++) {
    const wants = isStylusEraseRequest(pen(b));
    assert.equal(stylusEraseTransition(false, pen(b)), wants ? "engage" : "none", `mask ${b} from idle`);
    assert.equal(stylusEraseTransition(true, pen(b)), wants ? "none" : "release", `mask ${b} while borrowed`);
  }
});

test("a full press-hover-draw-release cycle ends back on the pen", () => {
  let borrowed = false;
  const step = (buttons: number) => {
    const a = stylusEraseTransition(borrowed, pen(buttons));
    if (a === "engage") borrowed = true;
    if (a === "release") borrowed = false;
    return a;
  };
  assert.equal(step(STYLUS_BARREL_BUTTON), "engage"); // hover, button down
  assert.equal(step(STYLUS_BARREL_BUTTON), "none"); // still hovering
  assert.equal(step(NIB | STYLUS_BARREL_BUTTON), "none"); // nib lands, erasing
  assert.equal(step(NIB | STYLUS_BARREL_BUTTON), "none"); // sweeping
  assert.equal(step(NIB), "release"); // button up, nib down
  assert.equal(borrowed, false, "the pen is back before the nib ever lifts");
  assert.equal(step(0), "none"); // lift off: nothing left to undo
});

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/stylus_button.test.ts ===
=====

=====
// === START FILE: tests/text_editing.test.ts ===
// File 187 of 194 | Lines: 323 | Size: 11868 bytes
=====

/**
 * Text boxes: started-but-empty boxes are thrown away, however editing ends.
 *
 * The bug this pins: tapping with the text tool creates a box straight away so
 * you can type. Tap away without typing and the box stayed forever, invisible
 * except for a "Type thoughts..." placeholder - and they piled up, because the
 * editor only deleted an empty box when called with a flag nothing ever set.
 */

```

```

* The fix lives in the STORE (a subscription on `editingId`), so it covers every
* way editing can end - Done, tapping away, switching tools, turning the page,
* switching board - including the dozen places that reset editingId directly.
*/

import assert from "node:assert/strict";

import { useWhiteboard } from "../src/store/whiteboard";
import { getBoardElements, allElements } from "../src/lib/whiteboard/notebook";
import { isAbandonedTextBox } from "../src/lib/whiteboard/textbox";
import {
  detectLineDir,
  prefixSelectedLines,
  resolveLineDir,
  setLineDirs,
  spliceFormatMarkers,
} from "../src/lib/whiteboard/textParser";
import { migrateState } from "../src/lib/whiteboard/persistence";
import type { Board, TextElement, WhiteboardElement } from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`${(err as Error).message}`);
  }
}

const S = () => useWhiteboard.getState();
const board = (id: string): Board => S().boards.find((b) => b.id === id)!;
const has = (boardId: string, elId: string) => allElements(board(boardId)).some((e) => e.id === elId);

let seq = 0;
function textBox(text = "", extra: Partial<TextElement> = {}): TextElement {
  seq++;
  return {
    id: `t${seq}_${Math.random().toString(36).slice(2, 6)}`,
    type: "text",
    x: 10 * seq,
    y: 10,
    width: 220,
    height: 48,
    rotation: 0,
    opacity: 1,
    text,
    color: "#1e293b",
    fontSize: 18,
    fontFamily: "Inter",
    fontWeight: 400,
    align: "left",
    createdAt: Date.now(),
    updatedAt: Date.now(),
    ...extra,
  } as TextElement;
}

function freshCanvas(): string {
  const id = S().createBoard(`C ${Math.random().toString(36).slice(2, 7)}`, null);
  S().setActiveBoard(id);
  return id;
}

function freshNotebook(): string {
  const id = S().createNotebook(`NB ${Math.random().toString(36).slice(2, 7)}`, null);
  S().setActiveBoard(id);
  return id;
}

/** What the canvas does on a text-tool tap: add, start editing, back to Select. */
function placeText(el: TextElement) {
  S().addElement(el, true);
  S().setEditingId(el.id);
  S().setTool("select");
}

```

```
}

console.log("\n■■■ Text boxes: abandoned-box clean-up ■■■");

test("an empty box is removed when editing ends", () => {
  const b = freshCanvas();
  const t = textBox();
  placeText(t);
  assert.ok(has(b, t.id), "the box exists while being edited");
  S().setEditingId(null);
  assert.equal(has(b, t.id), false, "tapping away with nothing typed removes it");
});

test("placing a box does NOT remove it (Select switch right after starting the edit)", () => {
  // The canvas starts the edit and switches to Select in the same breath. If
  // that tool switch ended editing, every new box would vanish immediately.
  const b = freshCanvas();
  const t = textBox();
  placeText(t);
  assert.equal(S().editingId, t.id);
  assert.ok(has(b, t.id));
  S().setEditingId(null);
});

test("a box with text is kept", () => {
  const b = freshCanvas();
  const t = textBox();
  placeText(t);
  S().updateElement(t.id, { text: "hello" } as Partial<TextElement>);
  S().setEditingId(null);
  assert.ok(has(b, t.id));
});

test("whitespace-only counts as empty", () => {
  const b = freshCanvas();
  const t = textBox();
  placeText(t);
  S().updateElement(t.id, { text: " \n\t " } as Partial<TextElement>);
  S().setEditingId(null);
  assert.equal(has(b, t.id), false);
});

test("switching to a drawing tool ends the edit (and cleans up)", () => {
  // Picking up a pen used to leave the editor open, still holding the keyboard.
  const b = freshCanvas();
  const t = textBox();
  placeText(t);
  S().setTool("pen");
  assert.equal(S().editingId, null, "the edit ended");
  assert.equal(has(b, t.id), false, "the empty box went with it");
});

test("starting a second box finishes the first", () => {
  const b = freshCanvas();
  const first = textBox();
  placeText(first);
  const second = textBox();
  placeText(second);
  assert.equal(has(b, first.id), false, "the untouched first box is gone");
  assert.ok(has(b, second.id), "the new one is being edited");
  S().setEditingId(null);
});

test("switching board while editing cleans up the box on the board you left", () => {
  const a = freshCanvas();
  const t = textBox();
  placeText(t);
  freshCanvas(); // setActiveBoard resets editingId directly
  assert.equal(has(a, t.id), false);
});

test("works inside a notebook page", () => {
  const nb = freshNotebook();
  const t = textBox();
  placeText(t);
  assert.ok(getBoardElements(board(nb)).some((e) => e.id === t.id));
  S().setEditingId(null);
  assert.equal(has(nb, t.id), false);
});
```

```

});

test("sticky notes are never auto-removed", () => {
  const b = freshCanvas();
  const sticky = { ...textBox(), type: "sticky", color: "yellow" } as unknown as WhiteboardElement;
  S().addElement(sticky, true);
  S().setEditingId(sticky.id);
  S().setEditingId(null);
  assert.ok(has(b, sticky.id));
});

test("an empty box styled as a shape (fill) is kept", () => {
  const b = freshCanvas();
  const t = textBox("", { backgroundColor: "#fde68a" });
  placeText(t);
  S().setEditingId(null);
  assert.ok(has(b, t.id));
});

console.log("\n■■■ Text boxes: undo after the clean-up ■■■");

test("no dead undo step is left behind", () => {
  // Placing the box pushed a pre-creation snapshot. Left there, the next undo
  // would visibly do nothing - the one right after would undo real work.
  const b = freshCanvas();
  const r = textBox("keep me");
  S().addElement(r, true);
  const depthBefore = S().history.past.length;
  const t = textBox();
  placeText(t);
  S().setEditingId(null);
  assert.equal(S().history.past.length, depthBefore, "the box's creation entry was removed");
  S().undo();
  assert.equal(has(b, r.id), false, "the FIRST undo now undoes the real last action");
});

test("an earlier real edit's undo entry is not eaten", () => {
  const b = freshCanvas();
  const r = textBox("moved");
  S().addElement(r, true);
  S().updateElement(r.id, { x: 999 } as Partial<TextElement>, true); // a real, undoable move
  const t = textBox();
  placeText(t);
  S().setEditingId(null);
  S().undo();
  const back = allElements(board(b)).find((e) => e.id === r.id!);
  assert.notEqual(back.x, 999, "undo still reverts the move");
  assert.ok(has(b, r.id), "and only the move");
});

console.log("\n■■■ Text boxes: loading older saves ■■■");

test("the loader drops boxes older builds left behind, and keeps real ones", () => {
  const saved = {
    version: 1,
    boards: [
      {
        id: "old",
        name: "Old board",
        folderId: null,
        elements: [textBox(""), textBox("  "), textBox("real text"), textBox("", { borderColor: "#000" })],
        camera: { x: 0, y: 0, zoom: 1 },
        createdAt: 1,
        updatedAt: 1,
      },
    ],
    folders: [],
    activeBoardId: "old",
  };
  const out = migrateState(saved as never);
  const els = out.boards[0].elements;
  assert.equal(els.length, 2, "two empty boxes dropped");
  assert.ok(els.some((e) => (e as TextElement).text === "real text"));
  assert.ok(els.some((e) => (e as TextElement).borderColor === "#000"), "a bordered box is a shape, kept");
});

test("format with no selection inserts empty markers, cursor between them", () => {
  assert.deepStrictEqual(spliceFormatMarkers("", 0, 0, "****", "****"), { updated: "*****", cursor: 2 });

```

```

    assert.deepEqual(spliceFormatMarkers("hi", 2, 2, "$", "$"), { updated: "hi$$", cursor: 3 });
    assert.deepEqual(spliceFormatMarkers("hi", 2, 2, "[[", "]]"), { updated: "hi[[ ]]", cursor: 4 });
  });

test("format with a selection still wraps it, cursor at its end", () => {
  assert.deepEqual(spliceFormatMarkers("hello", 1, 4, "***", "***"), { updated: "h**ell**o", cursor: 6 });
});

test("selection wrap holds mid-text, full-text and multi-char markers", () => {
  // Whole text selected.
  assert.deepEqual(spliceFormatMarkers("hi", 0, 2, "*", "*"), { updated: "**hi**", cursor: 3 });
  // Underline tags around one word mid-sentence.
  assert.deepEqual(spliceFormatMarkers("a b c", 2, 3, "<u>", "</u>"), {
    updated: "a <u>b</u> c",
    cursor: 6,
  });
  // WikiLink around a word.
  assert.deepEqual(spliceFormatMarkers("see Board now", 4, 9, "[[", "]]"), {
    updated: "see [[Board]] now",
    cursor: 11,
  });
  // Math around a word.
  assert.deepEqual(spliceFormatMarkers("area x here", 5, 6, "$", "$"), {
    updated: "area $$ here",
    cursor: 7,
  });
  // Empty markers mid-text park the cursor between them.
  assert.deepEqual(spliceFormatMarkers("ab", 1, 1, "***", "***"), { updated: "a****b", cursor: 3 });
});

test("line prefix with collapsed cursor keeps the single-line toggle", () => {
  assert.deepEqual(prefixSelectedLines("hello", 2, 2, "- "), { updated: "- hello", cursor: 7 });
  assert.deepEqual(prefixSelectedLines("- hello", 2, 2, "- "), { updated: "hello", cursor: 5 });
  assert.deepEqual(prefixSelectedLines("# Title", 0, 0, "## "), { updated: "## Title", cursor: 8 });
});

test("line prefix with a multi-line selection prefixes every touched line", () => {
  assert.deepEqual(prefixSelectedLines("a\nb", 0, 3, "- "), { updated: "- a\n- b", cursor: 7 });
  // All prefixed already: toggles off.
  assert.deepEqual(prefixSelectedLines("- a\n- b", 0, 5, "- "), { updated: "a\nb", cursor: 3 });
  // Mixed: the bare line gains it, the prefixed one keeps it.
  assert.deepEqual(prefixSelectedLines("- a\nb", 0, 5, "- "), { updated: "- a\n- b", cursor: 7 });
  // Selection ending at a line start does not claim that line.
  assert.deepEqual(prefixSelectedLines("a\nb", 0, 2, "- "), { updated: "- a\nb", cursor: 3 });
});

test("line direction auto-detects Hebrew/Arabic as RTL, Latin as LTR", () => {
  assert.equal(detectLineDir("■■■■ ■■■■"), "rtl");
  assert.equal(detectLineDir("■ ■■■■ ■■■■"), "rtl");
  assert.equal(detectLineDir("Hello world"), "ltr");
  assert.equal(detectLineDir("3. third item"), "ltr");
  assert.equal(detectLineDir("12 ■■■■"), "rtl");
  assert.equal(detectLineDir(""), "ltr");
  assert.equal(detectLineDir("123 !!!"), "ltr");
});

test("line direction override wins, missing means auto", () => {
  assert.equal(resolveLineDir(undefined, 0), "auto");
  assert.equal(resolveLineDir([], 0), "auto");
  assert.equal(resolveLineDir(["rtl"], 0), "rtl");
  assert.equal(resolveLineDir(["rtl"], 1), "auto");
  assert.equal(resolveLineDir(["auto"], 0), "auto");
});

test("direction sets every selected line and trims back to auto", () => {
  assert.deepEqual(setLineDirs(undefined, "a\nb", 0, 3, "rtl"), ["rtl", "rtl"]);
  assert.deepEqual(setLineDirs(["rtl", "rtl"], "a\nb", 0, 3, "auto"), [];
  assert.deepEqual(setLineDirs(undefined, "a\nb\nc", 2, 5, "rtl"), ["auto", "rtl", "rtl"]);
  // Partial selection still covers whole touched lines.
  assert.deepEqual(setLineDirs(undefined, "hello\nworld", 1, 8, "rtl"), ["rtl", "rtl"]);
});

test("isAbandonedTextBox is the single rule", () => {
  assert.equal(isAbandonedTextBox(textBox("")), true);
  assert.equal(isAbandonedTextBox(textBox("x")), false);
  assert.equal(isAbandonedTextBox(null), false);
  assert.equal(isAbandonedTextBox({ ...textBox(""), type: "sticky" } as unknown as WhiteboardElement), false);
});

```

```

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/text_editing.test.ts ===
=====

=====
// === START FILE: tests/toolbar_fixes.test.ts ===
// File 188 of 194 | Lines: 152 | Size: 5867 bytes
=====

/**
 * Tests for the Toolbar & Navigation Architecture Fixes:
 * - Shape stroke & fill color consistency in setToolInk and StyleState
 * - Sticky note color integration in getToolColor / setToolInk
 * - Context-aware undo/redo (canUndo/canRedo) scoped to active board
 * - Selection filter enforcement (passesSelectFilter) for single-click selection
 * - Store pagePanelOpen toggle, setter, and initial state
 * - Favorite pens bar pushOutOfRect screen boundary clamping
 */

import assert from "node:assert/strict";
import {
  DEFAULT_STYLE,
  setToolInk,
  getToolColor,
  type ToolType,
  passesSelectFilter,
  type SelectCategory,
  STICKY_COLORS,
} from "../src/lib/whiteboard/types";
import { useWhiteboard } from "../src/store/whiteboard";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(` ${err as Error}.message`);
  }
}

console.log(`\n■■■ Toolbar & Navigation Architecture Fixes ■■■`);

test("BUG-06 & BUG-07: setToolInk on shape tools sets shapeStrokeColor, shapeFillColor, and strokeColor", () => {
  const shapes: ToolType[] = ["rectangle", "ellipse", "diamond", "triangle", "star", "cloud"];
  for (const tool of shapes) {
    const patch = setToolInk(DEFAULT_STYLE, tool, { color: "#3b82f6" });
    assert.equal(
      (patch as { shapeStrokeColor?: string }).shapeStrokeColor,
      "#3b82f6",
      `${tool}: shapeStrokeColor must be set`
    );
    assert.equal(
      (patch as { shapeFillColor?: string }).shapeFillColor,
      "#3b82f6",
      `${tool}: shapeFillColor must follow color pick`
    );
    assert.equal(
      (patch as { strokeColor?: string }).strokeColor,
      "#3b82f6",
      `${tool}: strokeColor must be set for backward compatibility`
    );
  }
});

test("BUG-16: Sticky note color is properly read and updated via getToolColor and setToolInk", () => {
  const customStyle = { ...DEFAULT_STYLE, stickyColor: "pink" as const };
  assert.equal(getToolColor(customStyle, "sticky"), STICKY_COLORS.pink.bg);

```

```
const updated = setToolInk(DEFAULT_STYLE, "sticky", { color: "blue" });
assert.equal((updated as { stickyColor?: string }).stickyColor, "blue");

// Invalid color falls back to yellow
const fallback = setToolInk(DEFAULT_STYLE, "sticky", { color: "nonexistent_color" });
assert.equal((fallback as { stickyColor?: string }).stickyColor, "yellow");
});

test("BUG-08: passesSelectFilter strictly filters elements according to active filter category", () => {
  const strokeEl = { type: "stroke" as const };
  const textEl = { type: "text" as const };
  const stickyEl = { type: "sticky" as const };
  const rectEl = { type: "rectangle" as const };
  const starEl = { type: "star" as const };
  const imgEl = { type: "image" as const };

  const inkOnly = new Set<SelectCategory>(["ink"]);
  assert.equal(passesSelectFilter(strokeEl, inkOnly), true);
  assert.equal(passesSelectFilter(textEl, inkOnly), false);
  assert.equal(passesSelectFilter(stickyEl, inkOnly), false);
  assert.equal(passesSelectFilter(rectEl, inkOnly), false);
  assert.equal(passesSelectFilter(imgEl, inkOnly), false);

  const textAndShapes = new Set<SelectCategory>(["text", "shapes"]);
  assert.equal(passesSelectFilter(strokeEl, textAndShapes), false);
  assert.equal(passesSelectFilter(textEl, textAndShapes), true);
  assert.equal(passesSelectFilter(stickyEl, textAndShapes), true);
  assert.equal(passesSelectFilter(rectEl, textAndShapes), true);
  assert.equal(passesSelectFilter(starEl, textAndShapes), true);
  assert.equal(passesSelectFilter(imgEl, textAndShapes), false);
});

test("BUG-09: canUndo and canRedo are strictly scoped to the active board context", () => {
  const store = useWhiteboard.getState();
  const boardA = store.createBoard("Board A", null);
  const boardB = store.createBoard("Board B", null);

  // Switch to board A and commit an entry
  store.setActiveBoard(boardA);
  assert.equal(store.canUndo(), false, "Board A has no history yet");
  assert.equal(store.canRedo(), false);

  store.commitHistory();
  assert.equal(store.canUndo(), true, "Board A has undo after commit");

  // Switch to board B: should NOT see Board A's undo
  store.setActiveBoard(boardB);
  assert.equal(store.canUndo(), false, "Board B must not inherit Board A's undo history");

  // Switch back to Board A: undo is restored
  store.setActiveBoard(boardA);
  assert.equal(store.canUndo(), true, "Board A retains its undo history");
});

test("BUG-01: pagePanelOpen store property toggles and updates correctly", () => {
  const store = useWhiteboard.getState();
  store.setPagePanelOpen(false);
  assert.equal(useWhiteboard.getState().pagePanelOpen, false);

  store.togglePagePanel();
  assert.equal(useWhiteboard.getState().pagePanelOpen, true);

  store.togglePagePanel();
  assert.equal(useWhiteboard.getState().pagePanelOpen, false);

  store.setPagePanelOpen(true);
  assert.equal(useWhiteboard.getState().pagePanelOpen, true);
  store.setPagePanelOpen(false);
});

test("BUG-15: zoomTo(1) resets zoom level while preserving user pan coordinates (x, y)", () => {
  useWhiteboard.getState().setCamera({ x: 540, y: 720, zoom: 2.4 });
  assert.equal(useWhiteboard.getState().camera.zoom, 2.4);
  assert.equal(useWhiteboard.getState().camera.x, 540);
  assert.equal(useWhiteboard.getState().camera.y, 720);

  // Zooming to 100% via zoomTo(1) preserves the pan coordinates
```

```

    useWhiteboard.getState().zoomTo(1);
    const cam = useWhiteboard.getState().camera;
    assert.equal(cam.zoom, 1);
    assert.equal(cam.x, 540, "Pan X coordinate must not be clobbered");
    assert.equal(cam.y, 720, "Pan Y coordinate must not be clobbered");
  });

  console.log(`\n${passed} passed, ${failed} failed.\n`);
  if (failed > 0) {
    process.exit(1);
  }

  =====
  // === END FILE: tests/toolbar_fixes.test.ts ===
  =====

  =====
  // === START FILE: tests/toolbar_laser.test.ts ===
  // File 189 of 194 | Lines: 90 | Size: 3637 bytes
  =====

  /**
   * Customisable tool bar and the laser pointer's settings.
   * - A saved tool bar keeps only known, distinct buttons; junk means "default".
   * - The laser's colour and size live in their own fields, so changing them
   *   never touches a pen's ink or a shape's border.
   */

  import assert from "node:assert/strict";
  import {
    DEFAULT_STYLE,
    ALL_TOOLBAR_ITEMS,
    DEFAULT_TOOLBAR_ITEMS,
    cleanToolbarItems,
    mergeToolbarItems,
    getToolColor,
    getToolSize,
    setToolColor,
    setToolSize,
  } from "../src/lib/whiteboard/types";

  const NL = String.fromCharCode(10);
  let passed = 0;
  let failed = 0;
  function test(name: string, fn: () => void) {
    try {
      fn();
      passed++;
      console.log(` ✓ ${name}`);
    } catch (err) {
      failed++;
      console.error(` ✗ ${name}`);
      console.error(`   ${err as Error}.message`);
    }
  }

  console.log(NL + "■■■ Tool bar & laser ■■■");

  test("the default tool bar has the laser but not the hand, which can be added back", () => {
    assert.ok(DEFAULT_TOOLBAR_ITEMS.includes("laser"));
    assert.ok(!DEFAULT_TOOLBAR_ITEMS.includes("pan"));
    assert.ok(ALL_TOOLBAR_ITEMS.includes("pan"));
    assert.deepEqual(cleanToolbarItems(["pan", "pen"]), ["pan", "pen"]);
    assert.equal(new Set(ALL_TOOLBAR_ITEMS).size, ALL_TOOLBAR_ITEMS.length);
  });

  test("a saved tool bar keeps its order and drops unknown or repeated buttons", () => {
    assert.deepEqual(cleanToolbarItems(["text", "pen", "bogus", "pen", 3, "laser"]), ["text", "pen", "laser"]);
  });

  test("an empty or broken saved tool bar means the default", () => {
    assert.equal(cleanToolbarItems([]), undefined);
    assert.equal(cleanToolbarItems(["nope"]), undefined);
    assert.equal(cleanToolbarItems("pen"), undefined);
    assert.equal(cleanToolbarItems(undefined), undefined);
  });

```

```

test("laser colour and size are its own, not a pen's or a shape's", () => {
  const style = { ...DEFAULT_STYLE };
  const next = { ...style, ...setToolColor(style, "laser", "#22c55e"), ...setToolSize(style, "laser", 24) };
  assert.equal(getToolColor(next, "laser"), "#22c55e");
  assert.equal(getToolSize(next, "laser"), 24);
  assert.equal(next.strokeColor, style.strokeColor);
  assert.equal(next.strokeWidth, style.strokeWidth);
  assert.deepEqual(next.toolInk, style.toolInk);
});

test("a style saved before the laser existed still gives it a colour and size", () => {
  const old = { ...DEFAULT_STYLE } as Partial<typeof DEFAULT_STYLE>;
  delete old.laserColor;
  delete old.laserSize;
  assert.equal(getToolColor(old as typeof DEFAULT_STYLE, "laser"), DEFAULT_STYLE.laserColor);
  assert.equal(getToolSize(old as typeof DEFAULT_STYLE, "laser"), DEFAULT_STYLE.laserSize);
});

test("a tool added after the bar was saved appears; one the user removed does not", () => {
  // Saved before "Select text" existed (and without a removed-list).
  const old = DEFAULT_TOOLBAR_ITEMS.filter((id) => id !== "textselect");
  const merged = mergeToolbarItems(old, undefined);
  assert.ok(merged.includes("textselect"), "the new tool shows up");
  assert.ok(!merged.includes("pan"), "the hand stays off");
  assert.ok(merged.indexOf("textselect") < merged.indexOf("pen"), "at its default place");
  // Removed on purpose: stays removed.
  assert.ok(!mergeToolbarItems(old, ["textselect"]).includes("textselect"));
  // The user's own order is kept.
  assert.deepEqual(mergeToolbarItems(["text", "pen"], ALL_TOOLBAR_ITEMS.filter((i) => i !== "text" && i !== "pen")), ["text", "pen"]);
  assert.deepEqual(mergeToolbarItems(undefined, undefined), DEFAULT_TOOLBAR_ITEMS);
});

console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/toolbar_laser.test.ts ===
=====

=====
// === START FILE: tests/top_tools_and_sidebar.test.ts ===
// File 190 of 194 | Lines: 550 | Size: 21796 bytes
=====

/**
 * Automated Verification Suite:
 * Top-Right Tools Panel, Sidebar Sorting Engine & Pro Selection Actions
 */

import assert from "node:assert/strict";
import fs from "node:fs";
import path from "node:path";
import ts from "typescript";
import { sortItems } from "../src/lib/whiteboard/sort";
import { useWhiteboard } from "../src/store/whiteboard";
import type {
  StrokeElement,
  TextElement,
  StickyElement,
  RectangleElement,
  EllipseElement,
  CloudElement,
  ImageElement,
  ConnectionElement,
} from "../src/lib/whiteboard/types";

console.log("=== Testing Top-Right Tools Panel & Sidebar Sorting Engine ===");

function resetStore() {
  useWhiteboard.setState({
    boards: [],
    folders: [],
    activeBoardId: null,
    selectedIds: [],
    editingId: null,
    sidebarSortBy: "updated",
  });
}

```

```

    sidebarSortOrder: "desc",
    history: { past: [], future: [] },
  });
}

// -----
// 1. Sidebar Sorting Engine Tests (sortItems)
// -----
console.log("\n■■ 1. Sidebar Sorting Engine (sortItems) ■■");

{
  const items = [
    { name: "Note 10", createdAt: 1000, updatedAt: 5000 },
    { name: "Note 2", createdAt: 3000, updatedAt: 2000 },
    { name: "Note 1", createdAt: 2000, updatedAt: 8000 },
    { name: "Folder B", createdAt: 4000, // no updatedAt
  ];

  // Sort by Updated (desc - newest first)
  const byUpdatedDesc = sortItems(items, "updated", "desc");
  assert.equal(byUpdatedDesc[0].name, "Note 1", "Note 1 has latest updatedAt (8000)");
  assert.equal(byUpdatedDesc[1].name, "Note 10", "Note 10 has updatedAt 5000");
  assert.equal(byUpdatedDesc[2].name, "Folder B", "Folder B falls back to createdAt 4000");
  assert.equal(byUpdatedDesc[3].name, "Note 2", "Note 2 has updatedAt 2000");
  console.log(" ✓ sortItems: by 'updated' desc correctly uses updatedAt fallback to createdAt");

  // Sort by Updated (asc - oldest first)
  const byUpdatedAsc = sortItems(items, "updated", "asc");
  assert.equal(byUpdatedAsc[0].name, "Note 2", "Note 2 oldest (2000)");
  assert.equal(byUpdatedAsc[3].name, "Note 1", "Note 1 newest (8000)");
  console.log(" ✓ sortItems: by 'updated' asc correctly orders oldest first");

  // Sort by Created (desc - newest first)
  const byCreatedDesc = sortItems(items, "created", "desc");
  assert.equal(byCreatedDesc[0].name, "Folder B", "Folder B created newest (4000)");
  assert.equal(byCreatedDesc[1].name, "Note 2", "Note 2 created at 3000");
  assert.equal(byCreatedDesc[2].name, "Note 1", "Note 1 created at 2000");
  assert.equal(byCreatedDesc[3].name, "Note 10", "Note 10 created at 1000");
  console.log(" ✓ sortItems: by 'created' desc correctly orders newest createdAt first");

  // Sort by Created (asc - oldest first)
  const byCreatedAsc = sortItems(items, "created", "asc");
  assert.equal(byCreatedAsc[0].name, "Note 10", "Note 10 created oldest (1000)");
  assert.equal(byCreatedAsc[3].name, "Folder B", "Folder B created newest (4000)");
  console.log(" ✓ sortItems: by 'created' asc correctly orders oldest createdAt first");

  // Sort by Natural Name (A-Z)
  const byNameAsc = sortItems(items, "name", "asc");
  assert.equal(byNameAsc[0].name, "Folder B");
  assert.equal(byNameAsc[1].name, "Note 1");
  assert.equal(byNameAsc[2].name, "Note 2");
  assert.equal(byNameAsc[3].name, "Note 10", "Natural sort puts Note 2 before Note 10");
  console.log(" ✓ sortItems: by 'name' asc natural numeric sort (Note 2 before Note 10)");

  // Sort by Natural Name (Z-A)
  const byNameDesc = sortItems(items, "name", "desc");
  assert.equal(byNameDesc[0].name, "Note 10");
  assert.equal(byNameDesc[3].name, "Folder B");
  console.log(" ✓ sortItems: by 'name' desc correctly reverses natural alphabetical order");
}

// -----
// 2. Store Sort State Mutations
// -----
console.log("\n■■ 2. Store Sorting Actions (setSidebarSort) ■■");

{
  resetStore();
  const S = useWhiteboard.getState();

  assert.equal(S.sidebarSortBy, "updated");
  assert.equal(S.sidebarSortOrder, "desc");

  // Setting same field toggles direction
  S.setSidebarSort("updated");
  assert.equal(useWhiteboard.getState().sidebarSortOrder, "asc");

  S.setSidebarSort("updated");

```

```
assert.equal(useWhiteboard.getState().sidebarSortOrder, "desc");

// Switching field to "name" defaults to "asc"
S.setSidebarSort("name");
assert.equal(useWhiteboard.getState().sidebarSortBy, "name");
assert.equal(useWhiteboard.getState().sidebarSortOrder, "asc");

// Explicit order override
S.setSidebarSort("created", "desc");
assert.equal(useWhiteboard.getState().sidebarSortBy, "created");
assert.equal(useWhiteboard.getState().sidebarSortOrder, "desc");

console.log(" ✓ setSidebarSort updates sort field, toggles direction, and respects explicit order");
}

// -----
// 3. Pro Canvas Selection: selectByType
// -----
console.log("\n■ 3. Pro Canvas Selection (selectByType) ■");

{
  resetStore();
  const S = useWhiteboard.getState();
  S.createBoard("Test Selection Board", null);
  const activeBoard = useWhiteboard.getState().boards[0];

  const strokeEl: StrokeElement = {
    id: "str-1",
    type: "stroke",
    x: 0,
    y: 0,
    points: [[10, 10, 0.5]],
    color: "#000",
    strokeWidth: 2,
    strokeStyle: "ballpoint",
    opacity: 1,
    locked: false,
    rotation: 0,
    createdAt: 1,
    updatedAt: 1,
  };

  const lockedStrokeEl: StrokeElement = {
    id: "str-2",
    type: "stroke",
    x: 0,
    y: 0,
    points: [[20, 20, 0.5]],
    color: "#000",
    strokeWidth: 2,
    strokeStyle: "ballpoint",
    opacity: 1,
    locked: true,
    rotation: 0,
    createdAt: 1,
    updatedAt: 1,
  };

  const textEl: TextElement = {
    id: "txt-1",
    type: "text",
    x: 100,
    y: 100,
    width: 150,
    height: 40,
    text: "Hello World",
    fontSize: 16,
    fontFamily: "sans",
    fontWeight: 400,
    align: "left",
    color: "#000",
    opacity: 1,
    locked: false,
    rotation: 0,
    createdAt: 1,
    updatedAt: 1,
  };
}
```

```
const stickyEl: StickyElement = {
  id: "stk-1",
  type: "sticky",
  x: 200,
  y: 200,
  width: 120,
  height: 120,
  text: "Sticky Idea",
  fontSize: 14,
  color: "yellow",
  align: "center",
  opacity: 1,
  locked: false,
  rotation: 0,
  createdAt: 1,
  updatedAt: 1,
};

const rectEl: RectangleElement = {
  id: "shp-1",
  type: "rectangle",
  x: 300,
  y: 300,
  width: 100,
  height: 80,
  strokeColor: "#000",
  strokeWidth: 2,
  fillColor: "transparent",
  fillStyle: "solid",
  dashStyle: "solid",
  opacity: 1,
  locked: false,
  rotation: 0,
  createdAt: 1,
  updatedAt: 1,
};

const ellipseEl: EllipseElement = {
  id: "shp-2",
  type: "ellipse",
  x: 450,
  y: 300,
  width: 80,
  height: 80,
  strokeColor: "#000",
  strokeWidth: 2,
  fillColor: "transparent",
  fillStyle: "solid",
  dashStyle: "solid",
  opacity: 1,
  locked: false,
  rotation: 0,
  createdAt: 1,
  updatedAt: 1,
};

const cloudEl: CloudElement = {
  id: "shp-3",
  type: "cloud",
  x: 550,
  y: 300,
  width: 90,
  height: 60,
  strokeColor: "#000",
  strokeWidth: 2,
  fillColor: "transparent",
  fillStyle: "solid",
  dashStyle: "solid",
  opacity: 1,
  locked: false,
  rotation: 0,
  createdAt: 1,
  updatedAt: 1,
};

const imgEl: ImageElement = {
  id: "img-1",
  type: "image",
```

```
x: 600,
y: 200,
width: 200,
height: 150,
naturalWidth: 200,
naturalHeight: 150,
src: "",
opacity: 1,
locked: false,
rotation: 0,
createdAt: 1,
updatedAt: 1,
};

const connEl: ConnectionElement = {
  id: "con-1",
  type: "connection",
  x: 0,
  y: 0,
  source: { elementId: "shp-1", anchor: "right", boardId: activeBoard.id },
  target: { elementId: "shp-2", anchor: "left", boardId: activeBoard.id },
  linkType: "relates",
  routing: "curved",
  dashStyle: "solid",
  startArrowhead: "none",
  endArrowhead: "arrow",
  strokeWidth: 2,
  opacity: 1,
  locked: false,
  rotation: 0,
  createdAt: 1,
  updatedAt: 1,
};

S.addElement(strokeEl);
S.addElement(lockedStrokeEl);
S.addElement(textEl);
S.addElement(stickyEl);
S.addElement(rectEl);
S.addElement(ellipseEl);
S.addElement(cloudEl);
S.addElement(imgEl);
S.addElement(connEl);

// 3.1 Select by Stroke (should select str-1, skip locked str-2)
useWhiteboard.getState().selectByType("stroke");
const selectedStrokes = useWhiteboard.getState().selectedIds;
assert.deepEqual(selectedStrokes, ["str-1"], "Selects only unlocked stroke");
console.log(" ✓ selectByType('stroke'): selects unlocked ink strokes, ignores locked");

// 3.2 Select by Text
useWhiteboard.getState().selectByType("text");
assert.deepEqual(useWhiteboard.getState().selectedIds, ["txt-1"]);
console.log(" ✓ selectByType('text'): selects text cards");

// 3.3 Select by Sticky
useWhiteboard.getState().selectByType("sticky");
assert.deepEqual(useWhiteboard.getState().selectedIds, ["stk-1"]);
console.log(" ✓ selectByType('sticky'): selects sticky notes");

// 3.4 Select by Shape (rectangle, ellipse, cloud)
useWhiteboard.getState().selectByType("shape");
assert.deepEqual(useWhiteboard.getState().selectedIds.sort(), ["shp-1", "shp-2", "shp-3"].sort());
console.log(" ✓ selectByType('shape'): selects all shape elements (rectangles, ellipses, clouds)");

// 3.5 Select by Image
useWhiteboard.getState().selectByType("image");
assert.deepEqual(useWhiteboard.getState().selectedIds, ["img-1"]);
console.log(" ✓ selectByType('image'): selects images");

// 3.6 Select by Connection
useWhiteboard.getState().selectByType("connection");
assert.deepEqual(useWhiteboard.getState().selectedIds, ["con-1"]);
console.log(" ✓ selectByType('connection'): selects connection arrows");

// 3.7 Invert Selection
// When 'img-1' is selected, invertSelection() should select all other unlocked elements
useWhiteboard.getState().selectByType("image"); // img-1 selected
```

```

useWhiteboard.getState().invertSelection();
const inverted = useWhiteboard.getState().selectedIds.sort();
const expectedInverted = ["str-1", "txt-1", "stk-1", "shp-1", "shp-2", "shp-3", "con-1"].sort();
assert.deepEqual(inverted, expectedInverted, "Inverts selection to all other unlocked elements");
console.log(" ✓ invertSelection(): inverts selection to all unlocked unselected elements");

// Invert again should return back to ['img-1']
useWhiteboard.getState().invertSelection();
assert.deepEqual(useWhiteboard.getState().selectedIds, ["img-1"]);
console.log(" ✓ invertSelection(): involution property holds (inverting twice restores selection)");
}

// -----
// 4. Batch Operations (Move, Star, Delete)
// -----
console.log("\n■ 4. Batch Sidebar Operations (Move, Star, Delete) ■");

{
  resetStore();
  const S = useWhiteboard.getState();

  S.createFolder("Target Folder", null);
  const targetFolder = useWhiteboard.getState().folders[0];

  S.createBoard("Board Alpha", null);
  S.createBoard("Board Beta", null);
  S.createBoard("Board Gamma", null);
  const [bAlpha, bBeta, bGamma] = useWhiteboard.getState().boards;

  // 4.1 Batch Move
  useWhiteboard.getState().batchMoveBoards([bAlpha.id, bBeta.id], targetFolder.id);
  const boardsAfterMove = useWhiteboard.getState().boards;
  assert.equal(boardsAfterMove.find((b) => b.id === bAlpha.id)?.folderId, targetFolder.id);
  assert.equal(boardsAfterMove.find((b) => b.id === bBeta.id)?.folderId, targetFolder.id);
  assert.equal(boardsAfterMove.find((b) => b.id === bGamma.id)?.folderId, null);
  console.log(" ✓ batchMoveBoards: moves multiple boards into target folder atomically");

  // 4.2 Batch Star
  useWhiteboard.getState().batchStarBoards([bAlpha.id, bGamma.id], true);
  const boardsAfterStar = useWhiteboard.getState().boards;
  assert.equal(boardsAfterStar.find((b) => b.id === bAlpha.id)?.starred, true);
  assert.equal(boardsAfterStar.find((b) => b.id === bBeta.id)?.starred, false);
  assert.equal(boardsAfterStar.find((b) => b.id === bGamma.id)?.starred, true);

  useWhiteboard.getState().batchStarBoards([bAlpha.id, bGamma.id], false);
  const boardsAfterUnstar = useWhiteboard.getState().boards;
  assert.equal(boardsAfterUnstar.find((b) => b.id === bAlpha.id)?.starred, false);
  assert.equal(boardsAfterUnstar.find((b) => b.id === bGamma.id)?.starred, false);
  console.log(" ✓ batchStarBoards: stars and unstars multiple boards in a single action");

  // 4.3 Batch Delete
  // Add cross-board connection between Alpha and Gamma
  const conn: ConnectionElement = {
    id: "conn-cross",
    type: "connection",
    x: 0,
    y: 0,
    source: { elementId: "x", anchor: "top", boardId: bAlpha.id },
    target: { elementId: "y", anchor: "bottom", boardId: bGamma.id },
    linkType: "relates",
    routing: "straight",
    strokeWidth: 2,
    dashStyle: "solid",
    startArrowhead: "none",
    endArrowhead: "arrow",
    opacity: 1,
    rotation: 0,
    createdAt: 1,
    updatedAt: 1,
  };
  useWhiteboard.getState().addElement(conn);

  // Batch delete Alpha and Beta
  useWhiteboard.getState().batchDeleteBoards([bAlpha.id, bBeta.id]);
  const boardsAfterDelete = useWhiteboard.getState().boards;
  assert.equal(boardsAfterDelete.length, 1);
  assert.equal(boardsAfterDelete[0].id, bGamma.id);
  assert.equal(useWhiteboard.getState().activeBoardId, bGamma.id, "Selects remaining board as active");
}

```

```

// Connection should have been cleaned from bGamma because source was deleted
const gammaElements = boardsAfterDelete[0].elements;
assert.equal(gammaElements.some((el) => el.id === "conn-cross"), false, "Prunes dangling cross-board connections");
console.log(" ✓ batchDeleteBoards: deletes boards, cleans dangling connections, and maintains active board");
}

// -----
// 5. Sidebar Performance & Re-render Prevention Guardrails
// -----
console.log("\n■■■ 5. Sidebar Performance & Re-render Guardrails ■■■");

{
  const sidebarPath = path.resolve(__dirname, "../../src/components/whiteboard/Sidebar.tsx");
  const sidebarSrc = fs.readFileSync(sidebarPath, "utf-8");

  // Rule 1: BoardItem must be wrapped in React.memo(BoardItemImpl, boardItemPropsEqual)
  assert.ok(
    sidebarSrc.includes("React.memo(BoardItemImpl, boardItemPropsEqual)"),
    "BoardItem must be wrapped in React.memo with boardItemPropsEqual"
  );
  console.log(" ✓ BoardItem is wrapped in React.memo(BoardItemImpl, boardItemPropsEqual)");

  // Rule 1b: boardItemPropsEqual must check multiSelectMode and isSelected
  assert.ok(
    sidebarSrc.includes("a.multiSelectMode !== b.multiSelectMode") &&
    sidebarSrc.includes("a.isSelected !== b.isSelected"),
    "boardItemPropsEqual must compare multiSelectMode and isSelected"
  );
  console.log(" ✓ boardItemPropsEqual includes multiSelectMode and isSelected checks");

  // Rule 2: Sidebar must listen to boardListKey(s.boards) and NEVER s.boards directly
  assert.ok(
    sidebarSrc.includes("useWhiteboard((s) => boardListKey(s.boards))"),
    "Sidebar must subscribe to boardListKey(s.boards)"
  );
  assert.ok(
    !sidebarSrc.includes("const boards = useWhiteboard((s) => s.boards)"),
    "Sidebar must NEVER subscribe to s.boards directly"
  );
  console.log(" ✓ Sidebar subscribes via boardListKey(s.boards), avoiding s.boards direct subscription");

  // Rule 3: boardListKey must use sorted order of IDs and NEVER raw updatedAt timestamps
  assert.ok(
    sidebarSrc.includes(".sort((a, b) => (b.updatedAt || b.createdAt || 0) - (a.updatedAt || a.createdAt || 0))"),
    "boardListKey sorts boards to include ID order"
  );
  assert.ok(
    sidebarSrc.includes(".map((b) => b.id)"),
    "boardListKey extracts board IDs rather than timestamps"
  );
  assert.ok(
    !sidebarSrc.includes("b.updatedAt.toString()") && !sidebarSrc.includes("${b.updatedAt}"),
    "boardListKey does not serialize raw updatedAt timestamps"
  );
  console.log(" ✓ boardListKey incorporates sorted ID sequence, omitting raw updatedAt timestamps");

  // Rule 4: Functional validation of boardListKey stability & boardItemPropsEqual
  // Extract and evaluate the actual functions from Sidebar.tsx
  const keyFnMatch = sidebarSrc.match(/export function boardListKey\([^)]*\): string \{[\s\S]*?\n\}/);
  assert.ok(keyFnMatch, "boardListKey function definition found in Sidebar.tsx");
  const keyJs = ts.transpileModule(keyFnMatch[0], { compilerOptions: { module: ts.ModuleKind.CommonJS } }).outputText;
  const execBoardListKey = new Function("boards", "const exports = {}; " + keyJs + "; return exports.boardListKey(boards);");

  const equalFnMatch = sidebarSrc.match(/export function boardItemPropsEqual\([^)]*\): boolean \{[\s\S]*?\n\}/);
  assert.ok(equalFnMatch, "boardItemPropsEqual function definition found in Sidebar.tsx");
  const equalJs = ts.transpileModule(equalFnMatch[0], { compilerOptions: { module: ts.ModuleKind.CommonJS } }).outputText;
  const execBoardItemPropsEqual = new Function("a", "b", "const exports = {}; " + equalJs + "; return exports.boardItemPropsEqual(a, b);");

  // Test 5.1: Inking in active board preserves boardListKey identically across strokes

```

```

const boardA = { id: "nb-A", name: "Notebook A", folderId: null, starred: false, color: "#3b82f6", kind: "notebook" as const, tags: [], elements: [], createdAt: 1000, updatedAt: 5000 };
const boardB = { id: "nb-B", name: "Notebook B", folderId: null, starred: false, color: "#10b981", kind: "notebook" as const, tags: [], elements: [], createdAt: 1000, updatedAt: 2000 };
const initialKey = execBoardListKey([boardA, boardB]);

// Simulate 10 consecutive pen strokes drawn on active notebook A
for (let s = 1; s <= 10; s++) {
  const strokedA = {
    ...boardA,
    updatedAt: 5000 + s * 100,
    elements: [...boardA.elements, { id: `stroke-${s}`, type: "stroke" }]
  };
  const keyAfterStroke = execBoardListKey([strokedA, boardB]);
  assert.equal(keyAfterStroke, initialKey, `boardListKey must remain identical after stroke ${s} (prevents 2
    40ms pen-lift freeze)`);
}
console.log(" ✓ boardListKey remains strictly byte-identical across 10 inking strokes on active notebook");

// Test 5.2: Notebook ranking re-order correctly triggers key change
const bumpedB = { ...boardB, updatedAt: 9000 };
const keyAfterOrderChange = execBoardListKey([boardA, bumpedB]);
assert.notEqual(keyAfterOrderChange, initialKey, "boardListKey must update when sorted order of notebook IDs
  changes");
assert.ok(keyAfterOrderChange.startsWith("nb-B,nb-A"), "Updated notebook B moves to the front of sorted IDs
  sequence");
console.log(" ✓ boardListKey detects ranking re-order and updates sorted ID sequence");

// Test 5.3: boardItemPropsEqual memoization semantics
const baseItemProps = {
  board: boardA,
  isActive: true,
  depth: 0,
  multiSelectMode: false,
  isSelected: false,
  onClick: () => {},
};

// Adding strokes or bumping updatedAt does NOT invalidate BoardItem memoization
const strokedItemProps = {
  ...baseItemProps,
  board: { ...boardA, updatedAt: 6000, elements: [{ id: "str-1" }] }
};
assert.equal(execBoardItemPropsEqual(baseItemProps, strokedItemProps), true, "boardItemPropsEqual must return
  true on stroke updates");
console.log(" ✓ boardItemPropsEqual returns true when only elements or updatedAt change (no row re-render)
  ");

// Visual/selection props MUST invalidate BoardItem memoization
assert.equal(execBoardItemPropsEqual(baseItemProps, { ...baseItemProps, isSelected: true }), false, "Invalidated
  on isSelected");
assert.equal(execBoardItemPropsEqual(baseItemProps, { ...baseItemProps, multiSelectMode: true }), false, "Invalidated
  on multiSelectMode");
assert.equal(execBoardItemPropsEqual(baseItemProps, { ...baseItemProps, isActive: false }), false, "Invalidated
  on isActive");
assert.equal(execBoardItemPropsEqual(baseItemProps, { ...baseItemProps, depth: 1 }), false, "Invalidated on
  depth");
assert.equal(execBoardItemPropsEqual(baseItemProps, { ...baseItemProps, board: { ...boardA, name: "Renamed" }
  })), false, "Invalidated on name");
assert.equal(execBoardItemPropsEqual(baseItemProps, { ...baseItemProps, board: { ...boardA, starred: true }
  })), false, "Invalidated on starred");
assert.equal(execBoardItemPropsEqual(baseItemProps, { ...baseItemProps, board: { ...boardA, color: "#ef4444" }
  })), false, "Invalidated on color");
assert.equal(execBoardItemPropsEqual(baseItemProps, { ...baseItemProps, board: { ...boardA, folderId: "fld-1" }
  })), false, "Invalidated on folderId");
assert.equal(execBoardItemPropsEqual(baseItemProps, { ...baseItemProps, board: { ...boardA, tags: ["science"] }
  })), false, "Invalidated on tags");
console.log(" ✓ boardItemPropsEqual accurately detects all selection, multi-select, and metadata changes");
}

console.log("\n=====");
console.log("ALL TOP-RIGHT TOOLS & SIDEBAR SORTING TESTS PASSED (100%)");
console.log("=====");

=====
// === END FILE: tests/top_tools_and_sidebar.test.ts ===
=====

```

```

=====
// == START FILE: tests/wheel_device.test.ts ==
// File 191 of 194 | Lines: 210 | Size: 8213 bytes
=====

/**
 * Wheel device classification.
 *
 * The bug this pins: "sometimes when I drag down it zooms in instead".
 * `looksLikeMouseWheel` judged a single event, and a Windows precision
 * touchpad flicked straight down produces deltaX exactly 0 with an integer
 * deltaY over 50 – byte-for-byte what a wheel notch looks like. The same
 * gesture with a little sideways drift read as a trackpad. One gesture, two
 * answers, so the canvas zoomed at random.
 */

import assert from "node:assert/strict";

import {
  cameraPanForScroll,
  createWheelDeviceState,
  observeWheel,
  resolveWheelAction,
  wheelSourceOf,
  TRACKPAD_STREAM_GAP_MS,
  WHEEL_GESTURE_GAP_MS,
  type RawWheel,
  type WheelDeviceState,
} from "../src/lib/whiteboard/input";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`   ${err as Error}.message`);
  }
}

function ev(dy: number, dx = 0, deltaMode = 0): RawWheel {
  return { deltaX: dx, deltaY: dy, deltaMode, ctrlKey: false, metaKey: false, shiftKey: false };
}

/** Feeds a stream of events and returns the state plus the final source. */
function feed(
  frames: Array<{ dy: number; dx?: number; deltaMode?: number; gap?: number }>,
  start = 1000
): { state: WheelDeviceState; source: string } {
  let state = createWheelDeviceState();
  let t = start;
  let last = ev(0);
  for (const f of frames) {
    t += f.gap ?? 8; // touchpads stream at roughly 8-16ms
    last = ev(f.dy, f.dx ?? 0, f.deltaMode ?? 0);
    state = observeWheel(state, { deltaX: last.deltaX, deltaY: last.deltaY, deltaMode: last.deltaMode, time: t
    });
  }
  return { state, source: wheelSourceOf(state, last) };
}

console.log(`\n Wheel device classification `);

// --- the reported bug -----

test("a straight-down touchpad flick is NOT mistaken for a mouse wheel", () => {
  // Every frame looks exactly like a wheel notch on its own: dx=0, integer, >=50.
  const { source } = feed([
    { dy: 53 }, { dy: 61 }, { dy: 72 }, { dy: 66 }, { dy: 54 },
  ]);
  assert.equal(source, "trackpad", "the stream cadence gives the device away");
});

```

```

test("and that gesture pans instead of zooming", () => {
  const { state } = feed([{ dy: 53 }, { dy: 61 }, { dy: 72 }]);
  const action = resolveWheelAction(ev(72), "auto", wheelSourceOf(state, ev(72)));
  assert.equal(action.kind, "pan", "this is the zoom-instead-of-scroll bug");
});

test("one sideways-drifting frame settles the whole gesture as trackpad", () => {
  // The old per-event test flipped mid-gesture: drifting frames panned and
  // straight frames zoomed. The verdict must carry.
  const { source } = feed([{ dy: 40, dx: 3 }, { dy: 55 }, { dy: 60 }]);
  assert.equal(source, "trackpad");
});

test("a real mouse wheel still zooms", () => {
  // Discrete notches, far apart, no horizontal component.
  const { state } = feed([{ dy: 100, gap: 180 }, { dy: 100, gap: 200 }]);
  const action = resolveWheelAction(ev(100), "auto", wheelSourceOf(state, ev(100)));
  assert.equal(action.kind, "zoom");
  assert.ok(action.kind === "zoom" && action.factor < 1, "scrolling down zooms out");
});

test("a single isolated notch with no history is read as a wheel", () => {
  const state = createWheelDeviceState();
  assert.equal(wheelSourceOf(state, ev(120)), "mouse", "falls back to the one-event guess");
});

// --- decisive signals -----

test("line units mean a real wheel, whatever the cadence", () => {
  const { source } = feed([{ dy: 3, deltaMode: 1 }, { dy: 3, deltaMode: 1 }, { dy: 3, deltaMode: 1 }]);
  assert.equal(source, "mouse");
});

test("a fractional delta can only be a trackpad", () => {
  const { source } = feed([{ dy: 12.5, gap: 200 }]);
  assert.equal(source, "trackpad");
});

test("any horizontal movement means a trackpad", () => {
  const { source } = feed([{ dy: 80, dx: -2, gap: 300 }]);
  assert.equal(source, "trackpad");
});

// --- memory and re-decision -----

test("the verdict is held across the gesture, not recomputed each event", () => {
  let { state } = feed([{ dy: 10.5 }, { dy: 11 }]); // fraction proves trackpad
  // A later frame that looks exactly like a wheel notch must not flip it.
  state = observeWheel(state, { deltaX: 0, deltaY: 120, deltaMode: 0, time: 1100 });
  assert.equal(wheelSourceOf(state, ev(120)), "trackpad");
});

test("a long pause starts a new gesture and re-decides", () => {
  let { state } = feed([{ dy: 10.5 }, { dy: 11 }]);
  assert.equal(wheelSourceOf(state, ev(11)), "trackpad");

  // Switch devices: pause, then discrete notches.
  let t = 5000;
  for (let i = 0; i < 3; i++) {
    t += WHEEL_GESTURE_GAP_MS + 100;
    state = observeWheel(state, { deltaX: 0, deltaY: 100, deltaMode: 0, time: t });
  }
  assert.equal(wheelSourceOf(state, ev(100)), "mouse", "the old verdict must not stick forever");
});

test("events right at the stream threshold are not called a stream", () => {
  const { source } = feed([
    { dy: 100, gap: TRACKPAD_STREAM_GAP_MS },
    { dy: 100, gap: TRACKPAD_STREAM_GAP_MS },
    { dy: 100, gap: TRACKPAD_STREAM_GAP_MS },
  ]);
  assert.equal(source, "mouse", "the cadence rule is strictly below the threshold");
});

// --- settings still win -----

test("an explicit Mouse setting overrides the detector", () => {

```

```

const { state } = feed([ { dy: 12.5 }, { dy: 11.2 }, { dy: 10.9 } ]);
const action = resolveWheelAction(ev(12.5), "mouse", wheelSourceOf(state, ev(12.5)));
assert.equal(action.kind, "zoom");
});

test("an explicit Trackpad setting overrides the detector", () => {
  const { state } = feed([ { dy: 100, gap: 300 }, { dy: 100, gap: 300 } ]);
  const action = resolveWheelAction(ev(100), "trackpad", wheelSourceOf(state, ev(100)));
  assert.equal(action.kind, "pan");
});

test("a pinch always zooms, on either device", () => {
  const pinch: RawWheel = { ..ev(-8.5), ctrlKey: true };
  assert.equal(resolveWheelAction(pinch, "trackpad", "trackpad").kind, "zoom");
  assert.equal(resolveWheelAction(pinch, "auto", "trackpad").kind, "zoom");
});

// --- scroll direction -----
//
// `panCamera(dx, dy)` does `camera -= delta`: it takes a DRAG delta and moves
// the content with your hand. A scroll delta means the opposite, so these
// tests pin the conversion end to end – the bug was scrolling backwards.

/** What the camera's y becomes after scrolling by `deltaY`, from y = 0. */
function cameraYAfterScroll(deltaY: number, zoom = 1): number {
  const action = resolveWheelAction(ev(deltaY), "trackpad", "trackpad");
  assert.equal(action.kind, "pan", "trackpad mode must pan, not zoom");
  const pan = cameraPanForScroll(action as { kind: "pan"; dx: number; dy: number });
  // mirrors the store's panCamera
  return 0 - pan.dy / zoom;
}

test("scrolling DOWN moves the view down the page", () => {
  assert.ok(cameraYAfterScroll(120) > 0, "camera.y must increase, or the page scrolls backwards");
});

test("swiping UP on the touchpad does not move the view up", () => {
  // The reported bug, stated as the user hit it.
  assert.ok(cameraYAfterScroll(-120) < 0);
});

test("scrolling is symmetric: down then up returns to the start", () => {
  assert.equal(cameraYAfterScroll(120) + cameraYAfterScroll(-120), 0);
});

test("horizontal scroll follows the same convention", () => {
  const action = resolveWheelAction(
    { deltaX: 90, deltaY: 0, deltaMode: 0, ctrlKey: false, metaKey: false, shiftKey: false },
    "trackpad",
    "trackpad"
  );
  const pan = cameraPanForScroll(action as { kind: "pan"; dx: number; dy: number });
  assert.ok(0 - pan.dx > 0, "scrolling right shows content further right");
});

test("cameraPanForScroll is its own inverse", () => {
  const once = cameraPanForScroll({ dx: 7, dy: -3 });
  assert.deepEqual(cameraPanForScroll(once), { dx: 7, dy: -3 });
});

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/wheel_device.test.ts ===
=====

=====
// === START FILE: tests/workspace_store_v2.test.ts ===
// File 192 of 194 | Lines: 434 | Size: 16691 bytes
=====

/**
 * Tests for the IndexedDB workspace store (the localStorage-quota fix).
 *
 * Background: a tablet workspace grew past ~4 MB against localStorage's ~5 MB
 * per-origin hard cap, and EVERY save began failing with QuotaExceededError

```

```

* (dozens of "[whiteboard] Failed to save state" in the device logs). The fix
* moves the workspace snapshot to IndexedDB (disk-scale quota), gzipped past
* 1 MB, with the legacy localStorage path kept byte-identical for
* environments without IndexedDB (which is exactly what the E2E suites run).
*/

import assert from "node:assert/strict";
import {
  GZIP_THRESHOLD_CHARS,
  HEARTBEAT_KEY,
  STORAGE_PRESSURE_RATIO,
  createMemoryWorkspaceBackend,
  decodeWorkspace,
  encodeWorkspace,
  measureStoragePressure,
  requestPersistentStorage,
  type WorkspaceBackend,
  type WorkspaceRecord,
} from "../src/lib/whiteboard/workspaceDb";
import {
  cleanHistoryEntry,
  loadState,
  loadWorkspace,
  saveState,
  saveWorkspaceToBackend,
  type PersistedState,
} from "../src/lib/whiteboard/persistence";

let passed = 0;
let failed = 0;
async function atest(name: string, fn: () => Promise<void> | void) {
  try {
    await fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`${(err as Error).message}`);
  }
}

// -----
// Minimal window mock. Optionally capped like the ~5 MB WebView quota so the
// tablet failure can be replayed deterministically.
// -----
interface MockWindow {
  localStorage: {
    getItem(k: string): string | null;
    setItem(k: string, v: string): void;
    removeItem(k: string): void;
  };
  dispatchEvent(e: { type: string }): boolean;
}

function makeWindow(capChars = Infinity): { win: MockWindow; events: string[]; store: Map<string, string> } {
  const store = new Map<string, string>();
  const events: string[] = [];
  let used = 0;
  const win: MockWindow = {
    localStorage: {
      getItem: (k: string) => store.get(k) ?? null,
      setItem: (k: string, v: string) => {
        const prev = store.get(k);
        const next = used - (prev ? prev.length : 0) + v.length;
        if (next > capChars) {
          const err = new Error("Quota exceeded") as Error & { name: string };
          err.name = "QuotaExceededError";
          throw err;
        }
        store.set(k, v);
        used = next;
      },
      removeItem: (k: string) => {
        const prev = store.get(k);
        if (prev !== undefined) {
          used -= prev.length;
          store.delete(k);
        }
      },
    },
    dispatchEvent: (e) => {
      events.push(e.type);
      return true;
    },
  };
  return { win, events, store };
}

```

```

    },
  },
  dispatchEvent: (e: { type: string }) => {
    events.push(e.type);
    return true;
  },
};
return { win, events, store };
}

function setWindow(win: MockWindow | undefined) {
  const g = globalThis as Record<string, unknown>;
  if (win === undefined) delete g.window;
  else g.window = win;
}

type Point = [number, number, number];

function makeStroke(id: string, seed: number, points: number) {
  const pts: Point[] = [];
  for (let i = 0; i < points; i++) {
    pts.push([seed + i * 0.731, seed + Math.sin(i * 0.3) * 50, 0.5]);
  }
  return {
    id,
    type: "stroke",
    x: 0,
    y: 0,
    rotation: 0,
    opacity: 1,
    color: "#000000",
    strokeWidth: 2,
    strokeStyle: "ballpoint",
    points: pts,
    createdAt: 1,
    updatedAt: 1,
  };
}

/** A workspace of at least targetBytes of JSON – the tablet scenario. */
function makeBigState(targetBytes: number): PersistedState {
  const strokes: Array<ReturnType<typeof makeStroke>> = [];
  let n = 0;
  const base = {
    id: "big",
    name: "Big",
    folderId: null,
    camera: { x: 0, y: 0, zoom: 1 },
    createdAt: 1,
    updatedAt: 1,
    accessedAt: 1,
    tags: [],
    starred: false,
    color: "#3b82f6",
  };
  const probe = () => JSON.stringify(strokes).length;
  while (probe() < targetBytes) {
    strokes.push(makeStroke(`s${n}`, n, 200));
    n++;
  }
  const pageStroke = makeStroke("pageStroke", 999, 50);
  return {
    version: 1,
    folders: [{ id: "f1", name: "F", parentId: null, expanded: true, createdAt: 1, color: "#f59e0b" }],
    boards: [
      { ...base, elements: strokes },
      {
        ...base,
        id: "nb",
        name: "Nb",
        kind: "notebook",
        pageSize: "a4",
        pages: [{ id: "p1", elements: [pageStroke], createdAt: 1, updatedAt: 1 }],
        activePageId: "p1",
      },
    ],
    activeBoardId: "big",
  };
}

```

```

    activeFolderId: null,
  } as unknown as PersistedState;
}

function failingBackend(): WorkspaceBackend {
  const fail = () => Promise.reject(new Error("disk on fire"));
  return { load: fail, save: () => fail(), clear: fail };
}

async function main() {
  console.log("\n■■■ Workspace store v2 (IndexedDB quota fix) ■■■");

  await atest("codec stores small payloads raw and round-trips", async () => {
    const json = JSON.stringify({ hello: "world", n: 42 });
    const rec = await encodeWorkspace(json);
    assert.equal(rec.format, "raw");
    assert.equal(typeof rec.data, "string");
    assert.equal(await decodeWorkspace(rec), json);
  });

  await atest("codec gzips large payloads and round-trips", async () => {
    if (typeof CompressionStream === "undefined" || typeof DecompressionStream === "undefined") {
      console.log("    (no CompressionStream here – skipped)");
      return;
    }
    const json = JSON.stringify({ pts: "0123456789abcdef".repeat(GZIP_THRESHOLD_CHARS / 16) });
    assert.ok(json.length >= GZIP_THRESHOLD_CHARS, "payload must clear the gzip threshold");
    const rec = await encodeWorkspace(json);
    assert.equal(rec.format, "gzip");
    assert.ok(rec.data instanceof Uint8Array);
    assert.equal(await decodeWorkspace(rec), json);
  });

  await atest("gzip shrinks ink-like JSON to less than half", async () => {
    if (typeof CompressionStream === "undefined" || typeof DecompressionStream === "undefined") {
      console.log("    (no CompressionStream here – skipped)");
      return;
    }
    const strokes = [];
    for (let i = 0; i < 300; i++) strokes.push(makeStroke(`ink${i}`, i, 200));
    const json = JSON.stringify({ boards: [{ elements: strokes }] });
    assert.ok(json.length > GZIP_THRESHOLD_CHARS, "payload must clear the gzip threshold");
    const rec = await encodeWorkspace(json);
    assert.equal(rec.format, "gzip");
    const stored = (rec.data as Uint8Array).byteLength;
    assert.ok(stored / json.length < 0.5, `ratio ${stored / json.length} must be under 0.5`);
    assert.equal(await decodeWorkspace(rec), json);
  });

  await atest("decode rejects corrupt records instead of returning garbage", async () => {
    await assert.rejects(decodeWorkspace(null as unknown as WorkspaceRecord));
    await assert.rejects(decodeWorkspace({ v: 2, format: "raw", savedAt: 1, data: "{}" } as unknown as WorkspaceRecord));
    await assert.rejects(decodeWorkspace({ v: 1, format: "zip", savedAt: 1, data: "{}" } as unknown as WorkspaceRecord));
    await assert.rejects(
      decodeWorkspace({ v: 1, format: "raw", savedAt: 1, data: 42 } as unknown as WorkspaceRecord)
    );
    await assert.rejects(
      decodeWorkspace({ v: 1, format: "gzip", savedAt: 1, data: "not-bytes" } as unknown as WorkspaceRecord)
    );
    if (typeof DecompressionStream !== "undefined") {
      await assert.rejects(
        decodeWorkspace({ v: 1, format: "gzip", savedAt: 1, data: new Uint8Array([1, 2, 3, 4]) })
      );
    }
  });

  await atest("TABLET REPLAY: 6 MB workspace fails the legacy capped store but saves + reloads via the backend", async () => {
    const big = makeBigState(6_000_000);
    const rawLen = JSON.stringify(big).length;
    assert.ok(rawLen >= 6_000_000, `state must be >= 6 MB, got ${rawLen}`);

    // Old world: ~5 MB localStorage cap. The save (and its degraded retry)
    // both throw, wb:save-failed fires, and NOTHING is readable afterwards.
    const capped = makeWindow(5_000_000);
    setWindow(capped.win);
  });
}

```

```

    try {
      saveState(big);
      assert.ok(capped.events.includes("wb:save-failed"), "legacy path must report save-failed");
      assert.equal(loadState(), null, "legacy path must leave nothing readable");
    } finally {
      setWindow(undefined);
    }
  }

  // New world: the same bytes go through the backend uncapped.
  const backend = createMemoryWorkspaceBackend();
  const outcome = await saveWorkspaceToBackend(big, backend);
  assert.equal(outcome, "ok");
  const rec = backend.records.get("workspace");
  assert.ok(rec, "backend must hold the record");
  assert.equal(rec!.format, "gzip", "a 6 MB workspace must be stored gzipped");
  const { state: loaded, migratedFromLs } = await loadWorkspace(backend);
  assert.equal(migratedFromLs, false);
  assert.ok(loaded, "reloaded state must exist");
  // migrate() backfills defaults (e.g. bookmarked:false on pages), so the
  // comparison is at the content level: every folder, board and stroke.
  assert.equal(JSON.stringify(loaded!.folders), JSON.stringify(big.folders));
  assert.equal(loaded!.boards.length, 2);
  const loadedEls = (loaded!.boards[0] as unknown as { elements: unknown[] }).elements;
  const bigEls = (big.boards[0] as unknown as { elements: unknown[] }).elements;
  assert.equal(loadedEls.length, bigEls.length, "every stroke must survive the round-trip");
  assert.equal(JSON.stringify(loadedEls), JSON.stringify(bigEls));
  const loadedPages = (loaded!.boards[1] as unknown as { pages: Array<{ id: string; elements: unknown }> }).
    pages;
  const bigPages = (big.boards[1] as unknown as { pages: Array<{ id: string; elements: unknown }> }).pages;
  assert.equal(loadedPages.length, bigPages.length);
  assert.equal(loadedPages[0].id, bigPages[0].id);
  assert.equal(JSON.stringify(loadedPages[0].elements), JSON.stringify(bigPages[0].elements));
});

await atest("migration adopts the legacy localStorage copy and deletes it only after storing", async () => {
  const small = makeBigState(10_000);
  const ctx = makeWindow();
  setWindow(ctx.win);
  try {
    ctx.win.localStorage.setItem("z-whiteboard-state-v1", JSON.stringify(small));
    const backend = createMemoryWorkspaceBackend();
    const { state, migratedFromLs } = await loadWorkspace(backend);
    assert.equal(migratedFromLs, true);
    assert.ok(state, "migrated state must load");
    assert.equal(state!.boards.length, 2);
    assert.ok(backend.records.get("workspace"), "backend must hold the migrated record");
    assert.equal(
      ctx.win.localStorage.getItem("z-whiteboard-state-v1"),
      null,
      "legacy key must be deleted after a successful migration"
    );
  }
  // Second boot reads straight from the backend, no migration.
  const second = await loadWorkspace(backend);
  assert.equal(second.migratedFromLs, false);
  assert.equal(second.state!.boards.length, 2);
} finally {
  setWindow(undefined);
}
});

await atest("corrupt backend record falls back to localStorage and repairs the store", async () => {
  const small = makeBigState(10_000);
  const ctx = makeWindow();
  setWindow(ctx.win);
  try {
    ctx.win.localStorage.setItem("z-whiteboard-state-v1", JSON.stringify(small));
    const backend = createMemoryWorkspaceBackend();
    backend.records.set("workspace", { v: 1, format: "zip", savedAt: 1, data: "junk" } as unknown as Workspa
      ceRecord);
    const { state } = await loadWorkspace(backend);
    assert.ok(state, "must fall back to the localStorage copy");
    assert.equal(state!.boards.length, 2);
    // The fallback copy is written back, repairing the store – and what
    // lands there is the readable content, not the corruption.
    const repaired = backend.records.get("workspace")!;
    assert.ok(repaired.format === "raw" || repaired.format === "gzip");
    const repairedJson = JSON.parse(await decodeWorkspace(repaired)) as {
      boards: Array<{ elements?: unknown[]; pages?: Array<{ elements: unknown }> }>;

```

```

    };
    assert.equal(repairedJson.boards.length, 2);
    assert.equal(
      JSON.stringify(repairedJson.boards[0].elements ?? []),
      JSON.stringify((small.boards[0] as unknown as { elements: unknown }).elements ?? [])
    );
    assert.equal(repairedJson.boards[1].pages!.length, 1);
  } finally {
    setWindow(undefined);
  }
});

await atest("total backend failure reports failed and keeps the app alive", async () => {
  const ctx = makeWindow();
  setWindow(ctx.win);
  try {
    const small = makeBigState(10_000);
    const outcome = await saveWorkspaceToBackend(small, failingBackend());
    assert.equal(outcome, "failed");
    assert.ok(ctx.events.includes("wb:save-failed"));
  } finally {
    setWindow(undefined);
  }
});

await atest("image-heavy state that cannot fit degrades instead of dying", async () => {
  const ctx = makeWindow();
  setWindow(ctx.win);
  try {
    const withImage = makeBigState(10_000);
    const bigSrc = "data:image/png;base64," + "X".repeat(20_000);
    (withImage.boards[0] as unknown as { elements: unknown[] }).elements.push({
      id: "img1",
      type: "image",
      x: 0,
      y: 0,
      rotation: 0,
      opacity: 1,
      src: bigSrc,
      createdAt: 1,
      updatedAt: 1,
    });
    // Backend that only fits the image-stripped retry.
    const fullLen = JSON.stringify(withImage).length;
    const backend = createMemoryWorkspaceBackend();
    const picky: WorkspaceBackend = {
      load: () => backend.load(),
      clear: () => backend.clear(),
      save: async (rec: WorkspaceRecord) => {
        const len = typeof rec.data === "string" ? rec.data.length : rec.data.byteLength;
        if (len >= fullLen) throw new Error("QuotaExceededError");
        return backend.save(rec);
      },
    };
    const outcome = await saveWorkspaceToBackend(withImage, picky);
    assert.equal(outcome, "degraded");
    assert.ok(ctx.events.includes("wb:save-degraded"));
    const { state } = await loadWorkspace(picky);
    assert.ok(state, "degraded save must still reload");
    const img = (state!.boards[0] as unknown as { elements: Array<{ type: string; src?: string }> }).element
      s.find(
        (el) => el.type === "image"
      );
    assert.ok(img, "image element shell must survive");
    assert.equal(img!.src, "", "image pixels must be stripped in the degraded copy");
  } finally {
    setWindow(undefined);
  }
});

await atest("successful save stamps the cross-tab heartbeat", async () => {
  const ctx = makeWindow();
  setWindow(ctx.win);
  try {
    const outcome = await saveWorkspaceToBackend(makeBigState(10_000), createMemoryWorkspaceBackend());
    assert.equal(outcome, "ok");
    const beat = ctx.win.localStorage.getItem(HEARTBEAT_KEY);
    assert.ok(beat, "heartbeat key must be written");
  }
});

```

```

    assert.ok((JSON.parse(beat!) as { rev: number }).rev >= 1);
  } finally {
    setWindow(undefined);
  }
});

await atest("storage helpers honor the Storage API when present", async () => {
  const g = globalThis as Record<string, unknown>;
  const prevDesc = Object.getOwnPropertyDescriptor(g, "navigator");
  Object.defineProperty(g, "navigator", {
    value: {
      storage: {
        estimate: () => Promise.resolve({ usage: 9_000_000, quota: 10_000_000 }),
        persist: () => Promise.resolve(true),
      },
    },
    configurable: true,
    writable: true,
  });
  try {
    const pressure = await measureStoragePressure();
    assert.ok(pressure, "pressure must be measurable");
    assert.equal(pressure!.ratio, 0.9);
    assert.ok(pressure!.ratio >= STORAGE_PRESSURE_RATIO, "0.9 must trip the pressure line");
    assert.equal(await requestPersistentStorage(), true);
  } finally {
    if (prevDesc) Object.defineProperty(g, "navigator", prevDesc);
    else delete g.navigator;
  }
  assert.equal(await requestPersistentStorage(), false, "no Storage API means no persistence");
  assert.equal(await measureStoragePressure(), null, "no Storage API means no measurement");
});

await atest("legacy history entries still validate (untouched path)", async () => {
  assert.ok(
    cleanHistoryEntry({ boardId: "b", elements: [{ id: "e1", type: "stroke" }] }),
    "elements-only history entry must stay valid"
  );
});

console.log(`\n${passed} passed, ${failed} failed.`);
if (failed > 0) process.exit(1);
}

```

```
void main();
```

```
=====
// === END FILE: tests/workspace_store_v2.test.ts ===
=====
```

```
=====
// === START FILE: tsconfig.json ===
// File 193 of 194 | Lines: 42 | Size: 732 bytes
=====
```

```

{
  "compilerOptions": {
    "target": "ES2017",
    "lib": [
      "dom",
      "dom.iterable",
      "esnext"
    ],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ]
  }
}

```

```
    }
  ],
  "paths": {
    "@/*": [
      "./src/*"
    ]
  }
},
"include": [
  "**/*.ts",
  "**/*.tsx",
  ".next-dev/types/**/*.ts",
  ".next/dev/types/**/*.ts",
  ".next/types/**/*.ts",
  "next-env.d.ts"
],
"exclude": [
  "node_modules"
]
}
```

```
=====
// === END FILE: tsconfig.json ===
=====
```

```
=====
// === START FILE: tsconfig.test.json ===
// File 194 of 194 | Lines: 21 | Size: 402 bytes
=====
```

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "commonjs",
    "moduleResolution": "node",
    "esModuleInterop": true,
    "strict": true,
    "skipLibCheck": true,
    "noEmit": false,
    "outDir": ".test-dist",
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": [
    "src/lib/whiteboard/**/*.ts",
    "src/store/**/*.ts",
    "tests/**/*.ts"
  ]
}
```

```
=====
// === END FILE: tsconfig.test.json ===
=====
```